

Deep Learning 25/26

Master's Degree in Artificial Intelligence

Topic U6: Reinforcement Learning

Lecture 2: On-Policy Methods

2025/2026

David Olivieri
Leandro Rodríguez Liñares
Uvigo, E.S. Informatica

RL Lecture Schedule

Lecture 1: Foundations of Reinforcement Learning

1. Motivation: RL in Aerospace
2. Basic problem from examples; environments; build custom env;
3. Jump-start with DQN: seeing how it works
4. Elements of the V, Q and DQN
5. Practice: Develop custom Environment
6. Theoretical background: Dynamic programming and Bellman equation
7. TD and Monte Carlo
8. RL Concepts to problems

Lecture 2A: Policy-based methods and Actor-Critic

1. Recap of Qlearning:
2. Recap of DQN: More details of code
3. Practice: Develop custom Environment & the Rocket lander: The custom environment
4. When observations are pixels: Atari Games

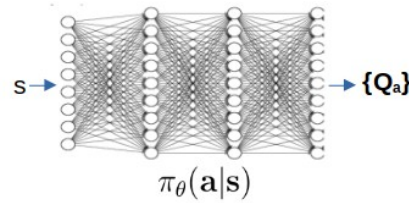
Lecture 2: On-Policy

1. Theory: Policy-based methods.
2. Implement Policy Gradient in pyTorch
3. The Actor -Critic Algorithm (A2C)
4. A2C in pyTorch
5. Comparison
6. Other On-policy and advanced methods
7. Training
8. Wing design with custom particle wind tunnel

Plan for today

1

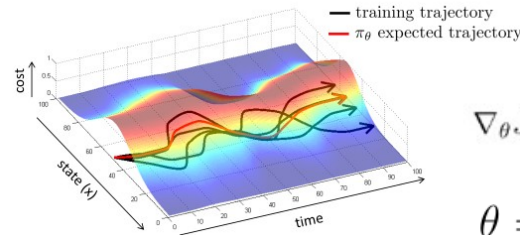
Recap and Beyond the DQN and off-policy



$$Q_{\phi}(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_{\phi}(\mathbf{s}', \mathbf{a}')$$

2

Theory: Policy-based Methods. Implement Policy Gradient in pyTorch

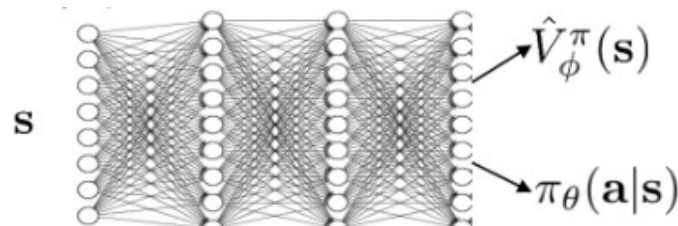


$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

3

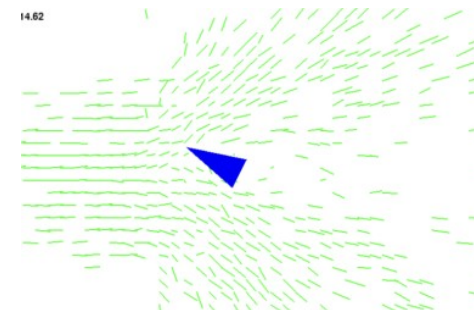
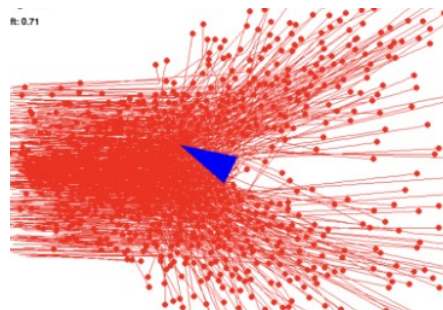
The Actor -Critic Algorithm (A2C)



$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \left(r + \gamma V_{\phi}(\mathbf{s}'_t) - V_{\phi}(\mathbf{s}_t) \right)$$

4

Example: Wing design with custom particle wind tunnel



Recap and Beyond the DQN and off-policy

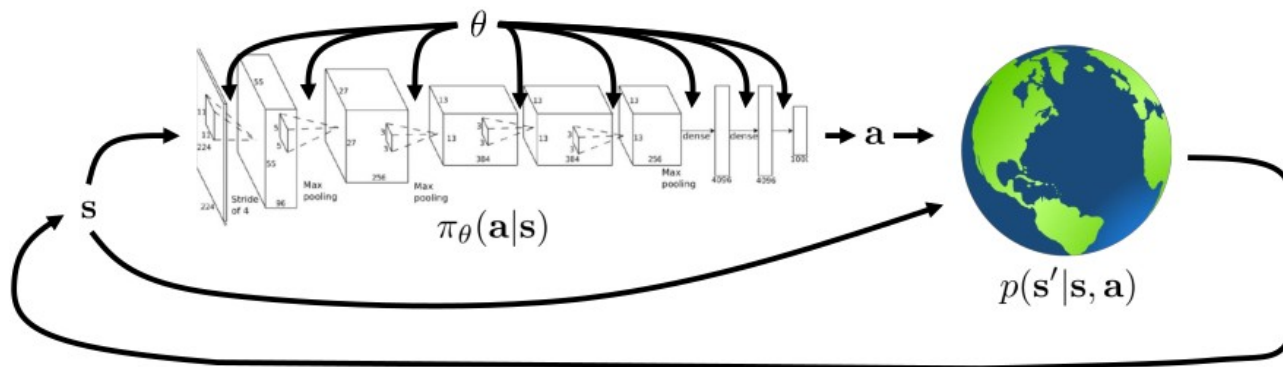
Recap of Q-Learning

full fitted Q-iteration algorithm:

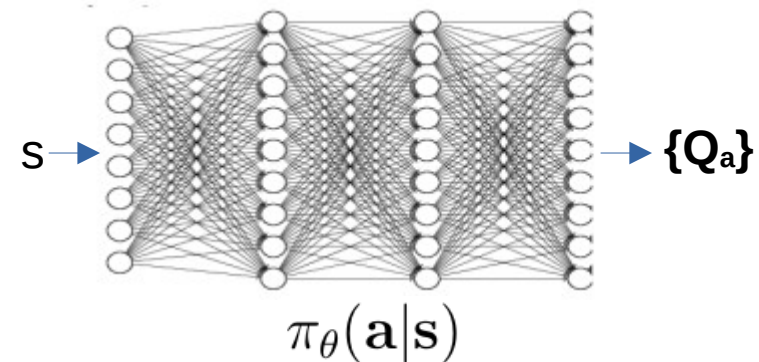
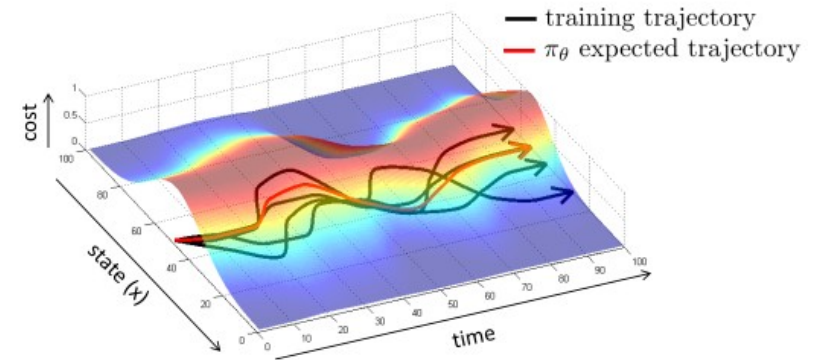
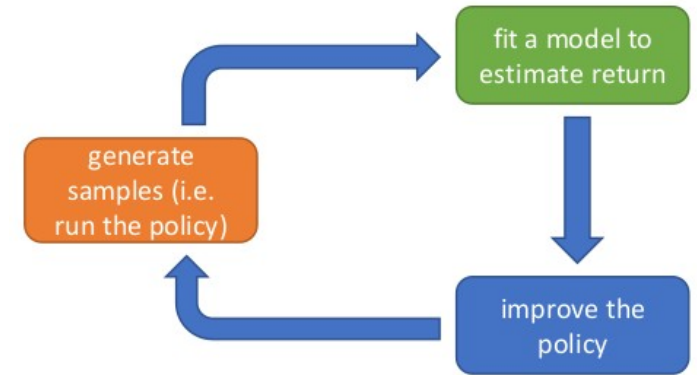
1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy
 2. set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}')$
 3. set $\phi \leftarrow \arg \min_{\phi} \frac{1}{2} \sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$
- $K \times$

online Q iteration algorithm:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}')$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$



$$Q_\phi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}', \mathbf{a}')$$



Reinforcement Learning Resources

Stable-Baselines3 assumes that you already understand the basic concepts of Reinforcement Learning (RL).

However, if you want to learn about RL, there are several good resources to get started:

- [OpenAI Spinning Up](#)
- [The Deep Reinforcement Learning Course](#)
- [David Silver's course](#)
- [Lilian Weng's blog](#)
- [Berkeley's Deep RL Bootcamp](#)
- [Berkeley's Deep Reinforcement Learning course](#)
- [DQN tutorial](#)
- [More resources](#)



<https://huggingface.co/learn/deep-rl-course/unit0/introduction>

Reinforcement Learning Summer School 2023 - DQN Tutorial

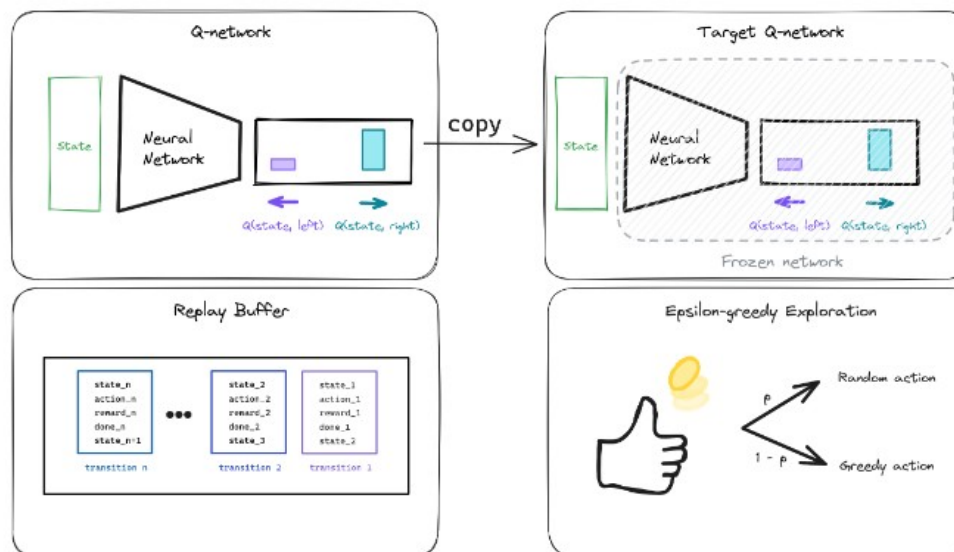
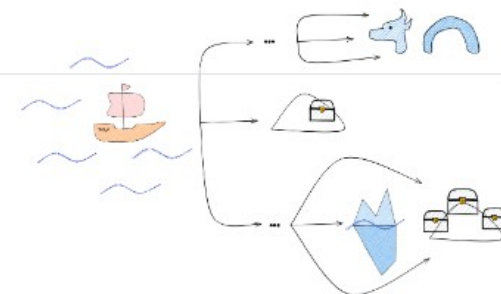
From Tabular Q-Learning to DQN

Website: <https://rlsummerschool.com/>

Slides: <https://araffin.github.io/slides/dqn-tutorial/>

Stable-Baselines3 repo: <https://github.com/DLR-RM/stable-baselines3>

RL Virtual School 2021: <https://github.com/araffin/rl-handson-rlvs21>



<https://github.com/araffin/rlss23-dqn-tutorial>

<https://rail.eecs.berkeley.edu/deeprlcourse-fa22/>



OpenAI Spinning Up

latest

USER DOCUMENTATION

Introduction

Installation

⊟ Algorithms

What's Included

⊟ Why These Algorithms?

⊟ Code Format

Running Experiments

Experiment Outputs

Plotting Results

INTRODUCTION TO RL

Part 1: Key Concepts in RL

Part 2: Kinds of RL Algorithms

Part 3: Intro to Policy Optimization

[Docs](#) » Algorithms

Algorithms

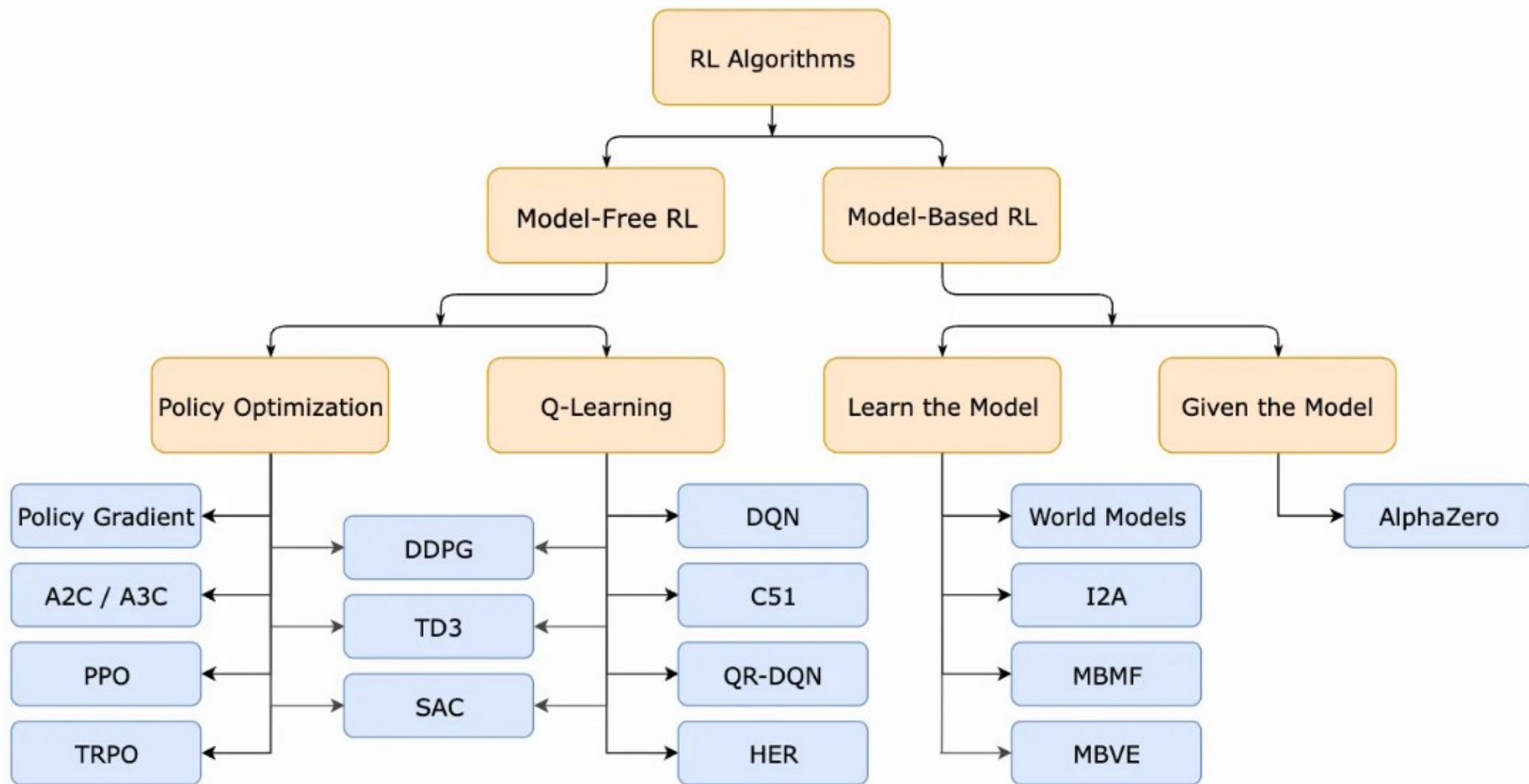
Table of Contents

- [Algorithms](#)
 - [What's Included](#)
 - [Why These Algorithms?](#)
 - [The On-Policy Algorithms](#)
 - [The Off-Policy Algorithms](#)
 - [Code Format](#)
 - [The Algorithm Function: PyTorch Version](#)
 - [The Algorithm Function: Tensorflow Version](#)
 - [The Core File](#)

What's Included

The following algorithms are implemented in the Spinning Up package:

- [Vanilla Policy Gradient \(VPG\)](#)
- [Trust Region Policy Optimization \(TRPO\)](#)
- [Proximal Policy Optimization \(PPO\)](#)
- [Deep Deterministic Policy Gradient \(DDPG\)](#)
- [Twin Delayed DDPG \(TD3\)](#)
- [Soft Actor-Critic \(SAC\)](#)



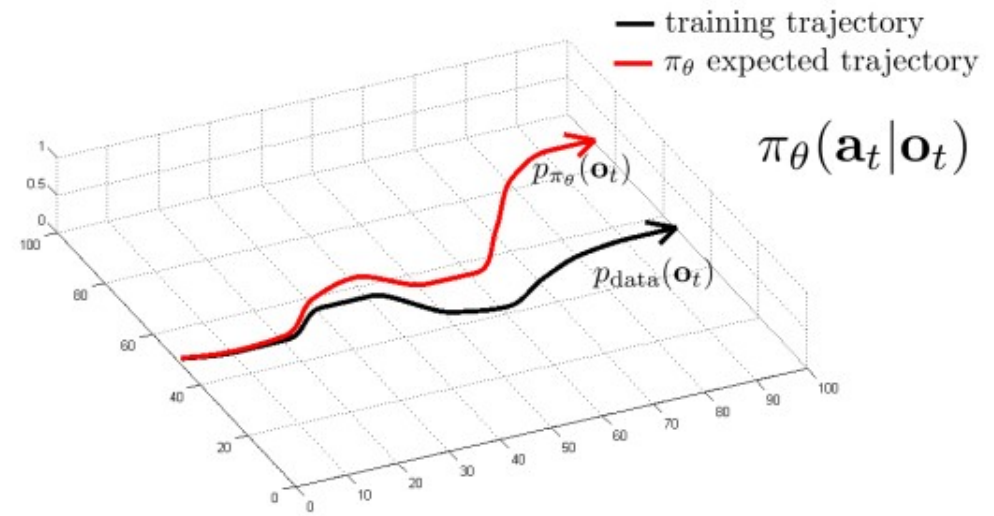
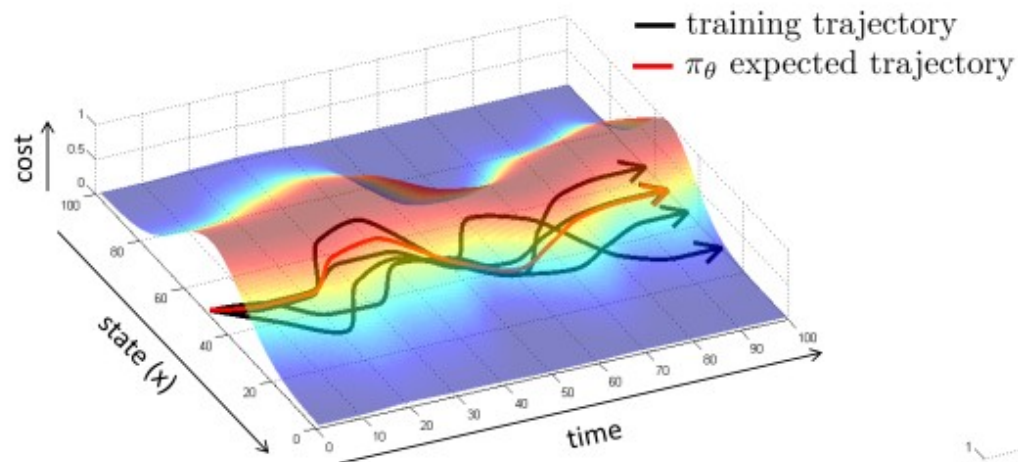
Taxonomy of modern RL algorithms (from [5])

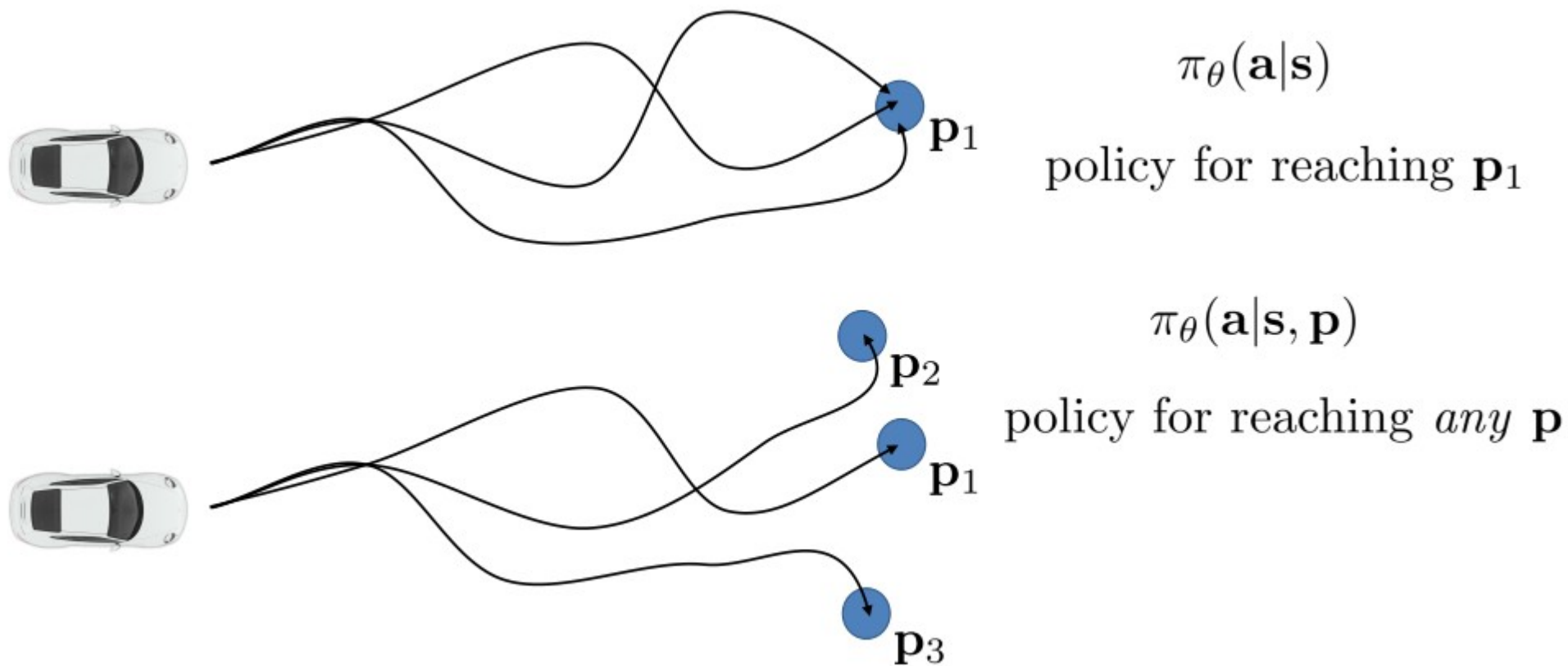
2

Theory: Policy-based
Methods. Implement Policy
Gradient in pyTorch

What are the policy-based methods?

- The main goal of Reinforcement learning is to find the optimal policy that will maximize the expected cumulative reward.
- reward hypothesis**: all goals can be described as the maximization of the expected cumulative reward.





Policy Gradient Overview

- instead of learning a Q-value function and using it to derive a policy, the agent directly learns a policy that maps states to action probabilities.
- allows the agent to sample actions directly from the policy's probability distribution and improve the policy over time.

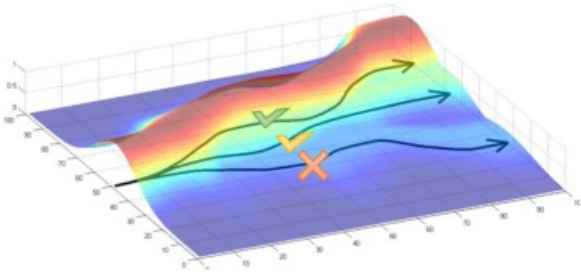
Key Differences from DQN

- **On-policy:** they learn from actions taken by the current policy (not from a replay buffer).
- **Action probabilities:** The agent learns a probability distribution over actions rather than estimating Q-values.
- **Continuous action spaces:** can handle continuous action spaces, unlike DQNs, which typically handle discrete actions.
- **No target network:** There's no need for a target network as the agent adjusts the policy based on each episode's outcome.

On-Policy Methods Overview

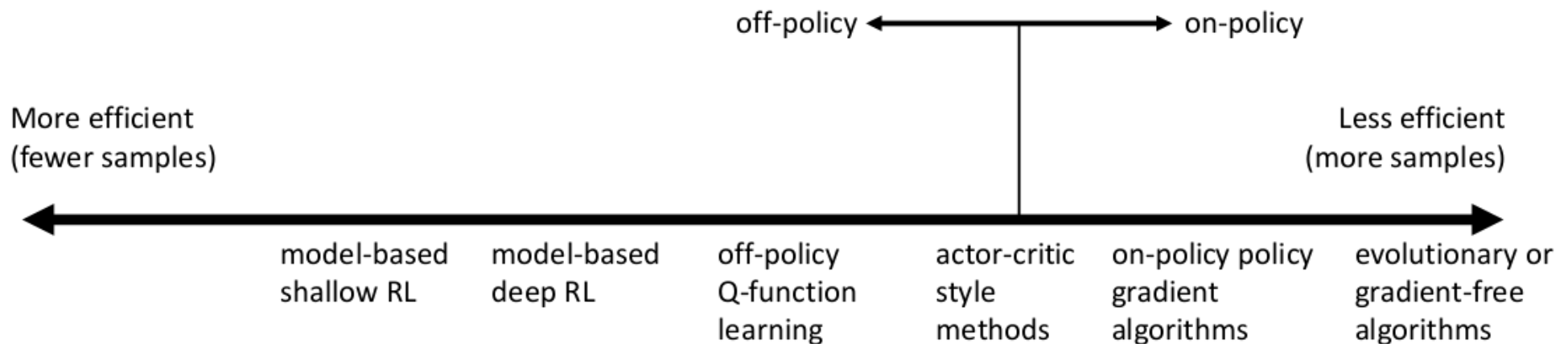
Definition:

- On-policy methods optimize and evaluate the same policy that is being used to generate the data (trajectories or episodes).



Key Characteristics:

- The agent interacts with the environment using its current policy π_θ
- Updates to the policy are based solely on trajectories sampled using the same policy.
- The focus is on improving the policy in small, incremental steps.



- Why would we use a less efficient algorithm?
- Wall clock time is not the same as efficiency!

- Does it converge?
- And if it converges, to what?
- And does it converge every time?

Advantages:

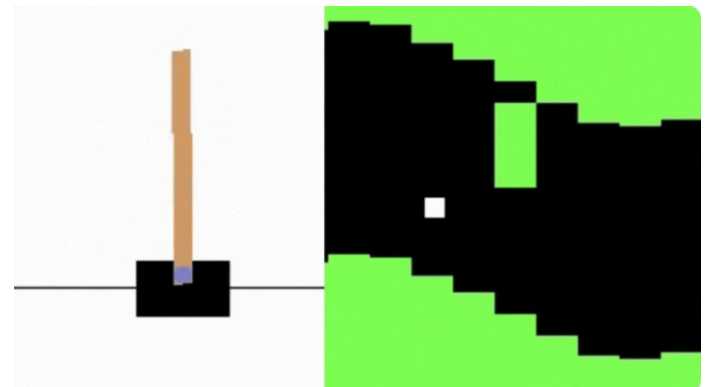
- Stable updates: Ensures that the updates align with the current policy's behavior.
- Exploration: Naturally incorporates exploration if the policy is stochastic.

Disadvantages:

- Sample inefficiency: Requires new samples for every policy update, discarding older data.
- High variance: Gradient estimates can have high variance, especially with long episodes

Common On-Policy Algorithms:

- REINFORCE (Monte Carlo policy gradient).
- Actor-Critic methods (e.g., A2C, PPO).
- Trust Region Policy Optimization (TRPO).



PixelCopter

Transition to Policy Gradient Methods

Motivation:

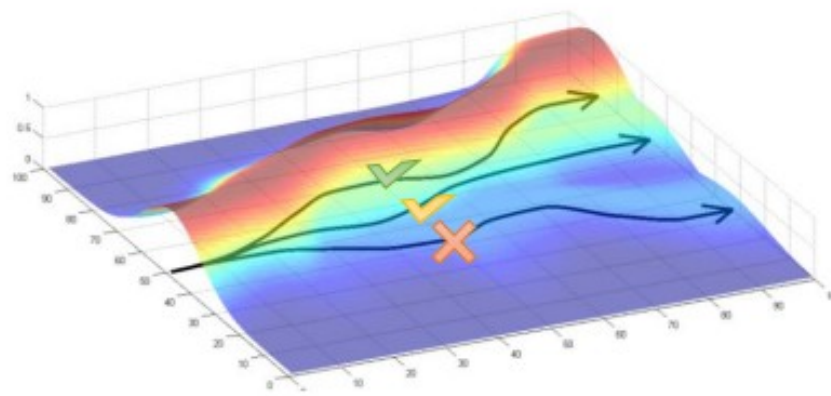
- In reinforcement learning, we aim to find a policy that maximizes the agent's total expected reward.
- Policies can be stochastic or deterministic:
- Stochastic policies: Map states to probabilities over actions.
- Deterministic policies: Map states to specific actions.
- For stochastic policies, the policy gradient method provides a direct way to optimize the policy.

Key Idea:

- Directly parameterize the policy $\pi_{\theta}(a|s)$ and adjust its parameters θ to maximize the expected return.
- Instead of learning a value function (like Q-learning), the focus is on learning the parameters of the policy directly.

Why Policy Gradient?:

- Works well with high-dimensional or continuous action spaces.
- Naturally incorporates stochasticity, which aids in exploration.
- Allows for flexible policy architectures (e.g., neural networks).



Symbol	Meaning
$s \in \mathcal{S}$	States.
$a \in \mathcal{A}$	Actions.
$r \in \mathcal{R}$	Rewards.
S_t, A_t, R_t	State, action, and reward at time step t of one trajectory. I may occasionally use s_t, a_t, r_t as well.
γ	Discount factor; penalty to uncertainty of future rewards; $0 < \gamma \leq 1$.
G_t	Return; or discounted future reward; $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.
$P(s', r s, a)$	Transition probability of getting to the next state s' from the current state s with action a and reward r .
$\pi(a s)$	Stochastic policy (agent behavior strategy); $\pi_{\theta}(\cdot)$ is a policy parameterized by θ .
$\mu(s)$	Deterministic policy; we can also label this as $\pi(s)$, but using a different letter gives better distinction so that we can easily tell when the policy is stochastic or deterministic without further explanation. Either π or μ is what a reinforcement learning algorithm aims to learn.
$V(s)$	State-value function measures the expected return of state s ; $V_w(\cdot)$ is a value function parameterized by w .
$V^{\pi}(s)$	The value of state s when we follow a policy π ; $V^{\pi}(s) = \mathbb{E}_{a \sim \pi}[G_t S_t = s]$.
$Q(s, a)$	Action-value function is similar to $V(s)$, but it assesses the expected return of a pair of state and action (s, a) ; $Q_w(\cdot)$ is a action value function parameterized by w .
$Q^{\pi}(s, a)$	Similar to $V^{\pi}(\cdot)$, the value of (state, action) pair when we follow a policy π ; $Q^{\pi}(s, a) = \mathbb{E}_{a \sim \pi}[G_t S_t = s, A_t = a]$.
$A(s, a)$	Advantage function, $A(s, a) = Q(s, a) - V(s)$; it can be considered as another version of Q-value with lower variance by taking the state-value off as the baseline.

General Form

- The goal of reinforcement learning is to find an optimal behavior strategy for the agent to obtain optimal rewards.
- The policy gradient methods target at modeling and optimizing the policy directly.
- The policy is usually modeled with a parameterized function

The Rewards Function

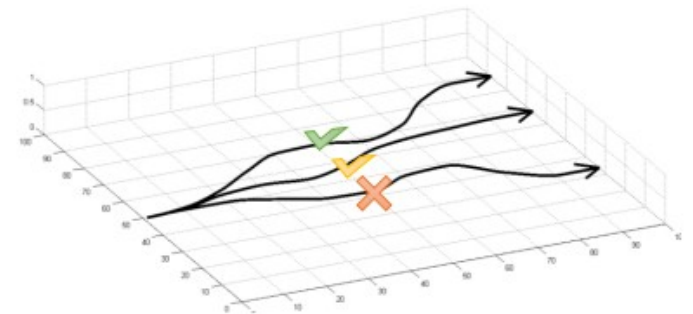
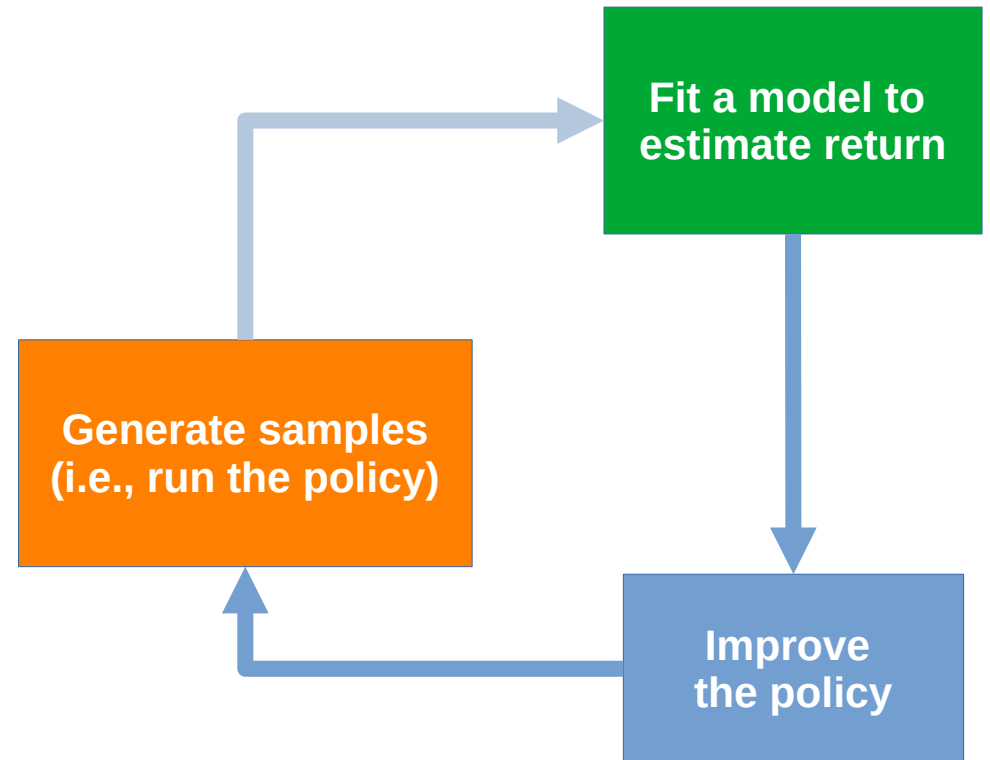
$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)]$$

Gradient for the update:

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

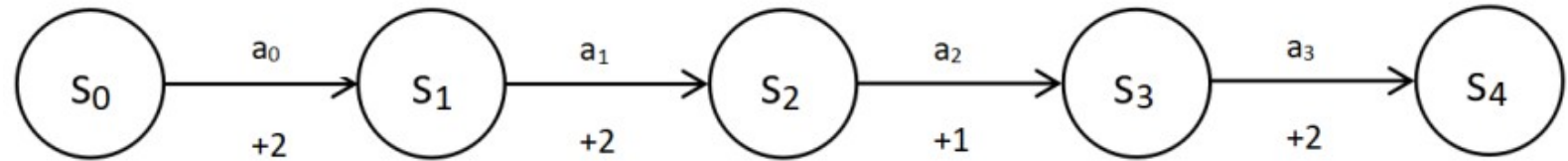
$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} [\nabla_{\theta} \log \pi_{\theta}(\tau) R(\tau)]$$

We will derive this later.



Return & discount factor

A particular “Episode”:



$$R(\tau) = r_0 + r_1 + r_2 + \dots + r_T$$

$$R(\tau) = \sum_{t=0}^T r_t$$

- **Return $R(\tau)$:** Cumulative reward over time
- **Goal:** Maximize total reward
 - Discount factor (γ):
 - Value: $0 \leq \gamma \leq 1$
- Represents preference for immediate rewards
 - Balances short-term and long-term rewards

introduce
discount factor

$$R(\tau) = \gamma^0 r_0 + \gamma^1 r_1 + \gamma^2 r_2 + \dots + \gamma^n r_\infty$$

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t$$

- Discount factor (γ) importance:
- Prevents return from reaching infinity
- Balances future and immediate rewards

Policy Gradient Algorithm (REINFORCE)

1. Initialize a policy $\pi_{\theta}(a|s)$

θ parameters of the policy (e.g., a neural network).

2. Collect an episode: interact with the environment using the current policy;

record each transition
 (s_t, a_t, r_t)

3. Calculate rewards for each step

The return is the cumulative discounted reward from step t

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

4. Compute the policy gradient:

The gradient of the objective function

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right]$$

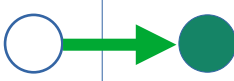
5. Update the policy parameters θ

Update in the direction of the gradient

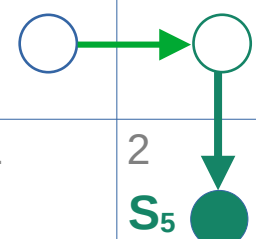
$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Episode: The agent transitions through the following states and actions

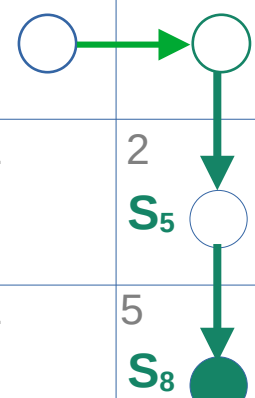
1	S₁	-1	S₂	-1
-1		2		-1
-1		5		+10



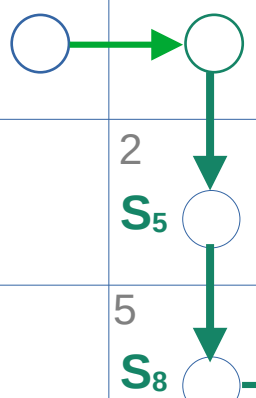
1	S₁	-1	S₂	-1
-1		2		-1
-1		5		+10



1	S₁	-1	S₂	-1
-1		2		-1
-1		5		+10



1	S₁	-1	S₂	-1
-1		2		-1
-1		5		+10



Calculate Return at each time step:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

$t = 0$ state 1

$$G_0 = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \gamma^4 r_4$$

$$G_0 = 1 + 0.9(-1) + 0.9^2(2) + 0.9^3(5) + 0.9^4(10)$$

$$G_0 = 1 - 0.9 + 1.62 + 3.645 + 6.561 = 11.926$$

$t = 1$ state 2

$$G_1 = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4$$

$$G_1 = -1 + 0.9(2) + 0.9^2(5) + 0.9^3(10)$$

$$G_1 = -1 + 1.8 + 4.05 + 7.29 = 12.14$$

$t = 2$ state 5

$$G_2 = r_2 + \gamma r_3 + \gamma^2 r_4$$

$$G_2 = 2 + 0.9(5) + 0.9^2(10)$$

$$G_2 = 2 + 4.5 + 8.1 = 14.6$$

$t = 3$ state 8

$$G_3 = r_3 + \gamma r_4$$

$$G_3 = 5 + 0.9(10)$$

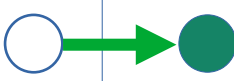
$$G_3 = 5 + 9 = 14$$

$t = 4$

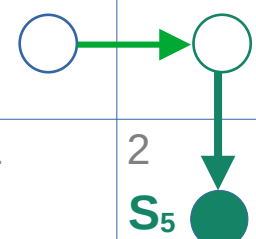
$$G_4 = r_4 = 10$$

Episode: The agent transitions through the following states and actions

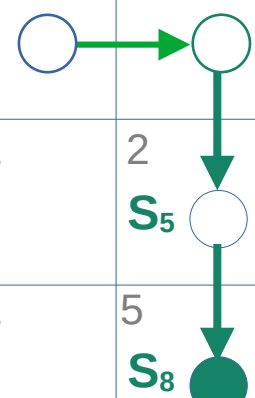
1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



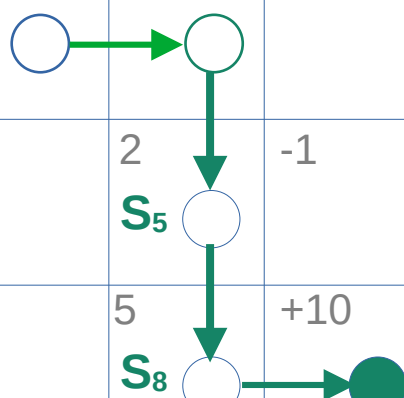
1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



A parameterized policy $\pi_{\theta}(a|s)$

Assume the following log-probability gradients:

$$\nabla_{\theta} \log \pi_{\theta}(a_1|1) = 0.1$$

$$\nabla_{\theta} \log \pi_{\theta}(a_2|2) = -0.2$$

$$\nabla_{\theta} \log \pi_{\theta}(a_3|5) = 0.3$$

$$\nabla_{\theta} \log \pi_{\theta}(a_4|8) = 0.5$$

$$t = 0 \quad \text{state 1} \quad G_0 \nabla_{\theta} \log \pi_{\theta}(a_1|1) = 11.926 \times 0.1 = 1.1926$$

$$t = 1 \quad \text{state 2} \quad G_1 \nabla_{\theta} \log \pi_{\theta}(a_2|2) = 12.14 \times (-0.2) = -2.428$$

$$t = 2 \quad \text{state 5} \quad G_2 \nabla_{\theta} \log \pi_{\theta}(a_3|5) = 14.6 \times 0.3 = 4.38$$

$$t = 3 \quad \text{state 8} \quad G_3 \nabla_{\theta} \log \pi_{\theta}(a_4|8) = 14 \times 0.5 = 7$$

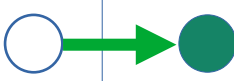
$$\Delta\theta = \alpha(1.1926 - 2.428 + 4.38 + 7)$$

Total Policy update:

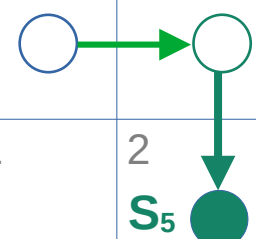
$$\Delta\theta = \alpha(10.1446)$$

Episode: The agent transitions through the following states and actions

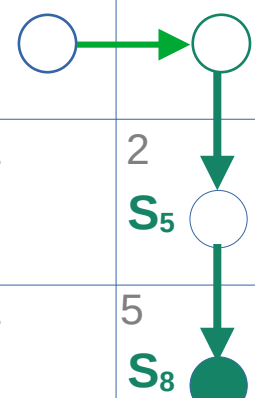
1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



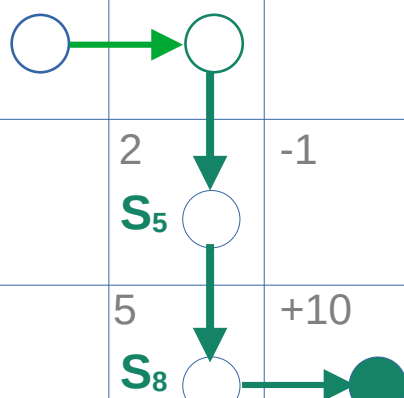
1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



2. Compute the Policy Gradient Update

$$\Delta\theta = \alpha \sum_{t=1}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

$$t = 1 \quad G_1 \nabla_{\theta} \log \pi_{\theta}(a_1 | 1) = 5.365 \times 0.1 = 0.5365$$

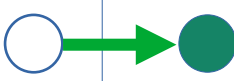
$$t = 2 \quad G_2 \nabla_{\theta} \log \pi_{\theta}(a_2 | 2) = 4.85 \times (-0.2) = -0.97$$

$$t = 3 \quad G_3 \nabla_{\theta} \log \pi_{\theta}(a_3 | 5) = 6.5 \times 0.3 = 1.95$$

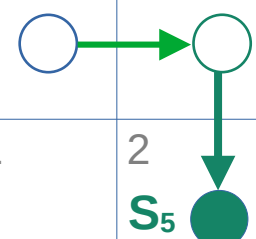
$$t = 4 \quad G_4 \nabla_{\theta} \log \pi_{\theta}(a_4 | 8) = 5 \times 0.5 = 2.5$$

Episode: The agent transitions through the following states and actions

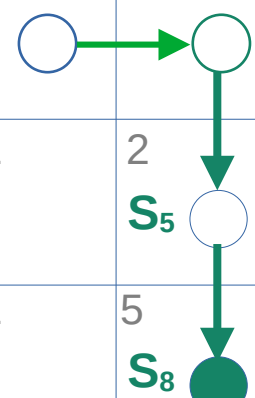
1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



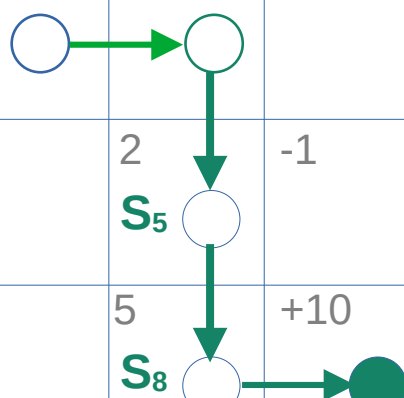
1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



1	s_1	-1	s_2	-1
-1		2		-1
-1		5		+10



3. Update the Policy Parameters θ

learning rate: $\alpha = 0.01$

$$\Delta\theta = \alpha(1.1926 - 2.428 + 4.38 + 7)$$

$$\Delta\theta = \alpha(10.1446) = 0.101446$$

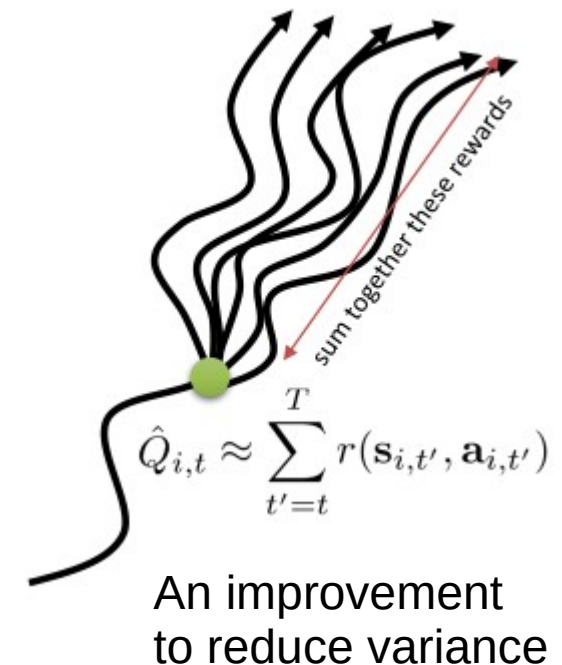
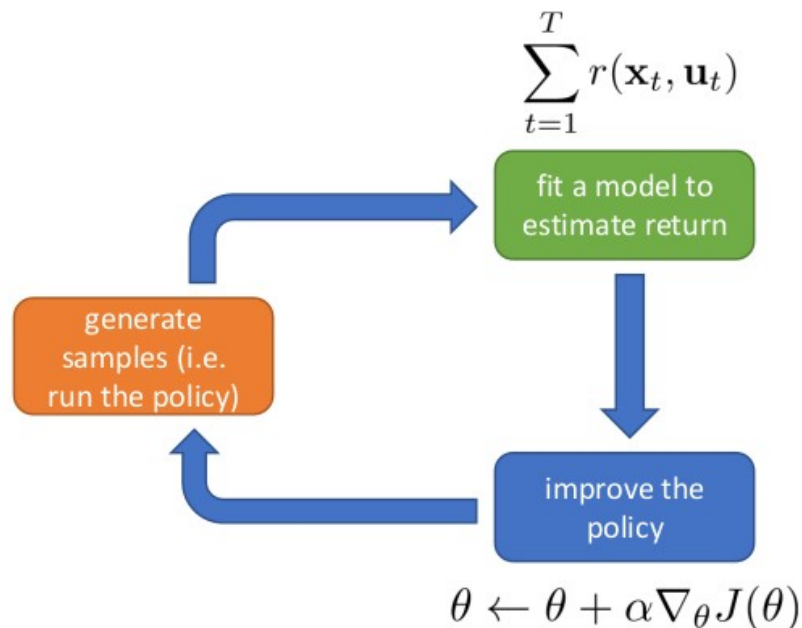
Update to parameters:

$$\theta_{\text{new}} = \theta_{\text{old}} + 0.101446$$

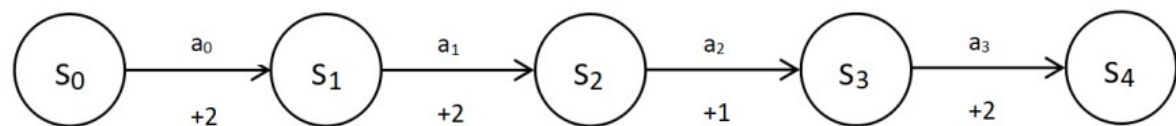
Deriving the Policy Gradient

On-policy:

- REINFORCE collects entire episodes, computes returns, and directly updates the policy using the episode's information.
- It does not rely on a replay buffer; instead, it processes each episode sequentially, updating after each full episode with a gradient based on the episode's total reward.
- This makes the method on-policy, as it depends directly on actions taken by the current policy in the present environment.



Deriving Policy Gradient

1. Define the Objective $J(\theta)$ 

$$R(\tau) = r_0 + r_1 + r_2 + \dots + r_T$$

$$R(\tau) = \sum_{t=0}^T r_t$$

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)]$$

τ Represents a trajectory or sequence of states, actions, and rewards $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$ generated by a policy

$R(\tau)$ is the cumulative reward (return) for the trajectory.

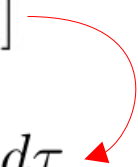
π_{θ} is the policy parameterized by θ

The expectation

$\mathbb{E}_{\tau \sim \pi_{\theta}}$ averaging the total return over all possible trajectories τ generated by the policy π_{θ}

Deriving Policy Gradient

2. Take the derivative: Use the Log-Trick

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$
$$J(\theta) = \int \pi_\theta(\tau) R(\tau) d\tau$$


Take Gradient:

$$\begin{aligned} \nabla_\theta J(\theta) &= \int \nabla_\theta \pi_\theta(\tau) R(\tau) d\tau \\ &= \int \nabla_\theta \pi_\theta(\tau) \frac{\pi_\theta(\tau)}{\pi_\theta(\tau)} R(\tau) d\tau \\ &= \int \pi_\theta(\tau) \left[\frac{\nabla_\theta \pi_\theta(\tau)}{\pi_\theta(\tau)} \right] R(\tau) d\tau \\ &= \int \pi_\theta(\tau) \nabla_\theta \log \pi_\theta(\tau) R(\tau) d\tau \\ \nabla_\theta J(\theta) &= \mathbb{E}_{\tau \sim \pi_\theta(\tau)} [\nabla_\theta \log \pi_\theta(\tau) R(\tau)] \end{aligned}$$


Monte Carlo

$$\mathbb{E}_{\tau \sim \pi_\theta(\tau)} \approx \underbrace{\frac{1}{N} \sum_{i=0}^N}_{\text{Sum over } N \text{ trajectories}} \left[\text{Ensemble of trajectories} \right]$$

The log-trick is used to convert the original Integral into an expectation;

This is easily solved with the Monte Carlo

2. Expand the Expectation over Trajectories

$$\pi_{\theta}(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t|s_t)p(s_{t+1}|s_t, a_t)$$

The probability of a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$

under the policy π_{θ}

is given by the product of probabilities of each state-action pair:

$$P(\tau|\theta) = p(s_0) \prod_{t=0}^T \pi_{\theta}(a_t|s_t)p(s_{t+1}|s_t, a_t)$$

$p(s_0)$ is the probability of the initial state.

$\pi_{\theta}(a_t|s_t)$ is the probability of taking action a_t in state s_t under policy π_{θ}

$p(s_{t+1}|s_t, a_t)$ is the transition probability to s_{t+1} given state s_t and action a_t

2. Expand the Expectation over Trajectories

$$\begin{aligned}\pi_{\theta}(\tau) &= p(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t|s_t)p(s_{t+1}|s_t, a_t) \\ \log \pi_{\theta}(\tau) &= \log \left[p(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t|s_t)p(s_{t+1}|s_t, a_t) \right] \\ &= \log p(s_0) + \log \prod_{t=0}^{T-1} [\pi_{\theta}(a_t|s_t)p(s_{t+1}|s_t, a_t)] \\ &= \log p(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t|s_t) + \log p(s_{t+1}|s_t, a_t)\end{aligned}$$

2. Expand the Expectation over Trajectories

$$\log \pi_{\theta}(\tau) = \log p(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t) + \log p(s_{t+1} | s_t, a_t)$$

$$\nabla_{\theta} \log \pi_{\theta}(\tau) = \nabla_{\theta} \left(\log p(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t) + \log p(s_{t+1} | s_t, a_t) \right)$$

$$\nabla_{\theta} \log \pi_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

2. Substitute into policy Gradient expression

$$\nabla_{\theta} \log \pi_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} [\nabla_{\theta} \log \pi_{\theta}(\tau) R(\tau)]$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

The Expectation
We can approximate
With a Monte Carlo average

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

6. Substitute back into the Gradient Expression

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) R(\tau) \right] \quad \text{policy gradient formula}$$

- shows that the gradient of the expected return with respect to the policy parameters θ is an expectation over trajectories.
- each term $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ scales with the cumulative reward $R(\tau)$ of the trajectory, encouraging actions that led to higher rewards.

Deriving Policy Gradient

7. Use a discounted return

$$R(\tau) \longrightarrow G_t$$

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

provides a more localized feedback signal and often reduces the variance of the gradient estimates

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

How do we actually compute this?

Deriving Policy Gradient

7. Use a discounted return

$$R(\tau) \longrightarrow G_t$$

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

provides a more localized feedback signal and often reduces the variance of the gradient estimates

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

Use the Monte Carlo method to approximate the expectation using the average

$$\nabla_{\theta} J(\theta) = \underbrace{\frac{1}{N} \sum_{i=0}^N}_{\text{Sum over } N \text{ trajectories}} \left[\sum_{t=0}^T \underbrace{G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)}_{\text{For each step in the trajectory}} \right]$$

collect N number of trajectories $\{\tau^i\}_{i=0}^N$ following policy: π_{θ}

Step-by-Step Training Algorithm for REINFORCE

1. Initialize the network parameters θ with random values
2. Generate N trajectories $\{\tau^i\}_{i=0}^N$ following policy: π_θ
3. Compute the return $R(\tau)$ or the discounted return G_t
4. Compute the gradients:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau) \right]$$

5. Update the network parameter as

$$\theta = \theta + \alpha \nabla_\theta J(\theta)$$

6. Repeat steps 2 to 5 for several iterations

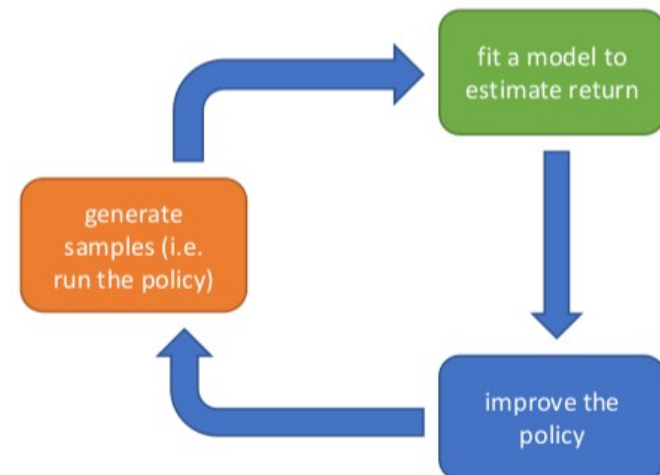
7. Use a discounted return

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{i=0}^N \left[\sum_{t=0}^T \underbrace{G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)}_{\text{For each step in the trajectory}} \right]$$

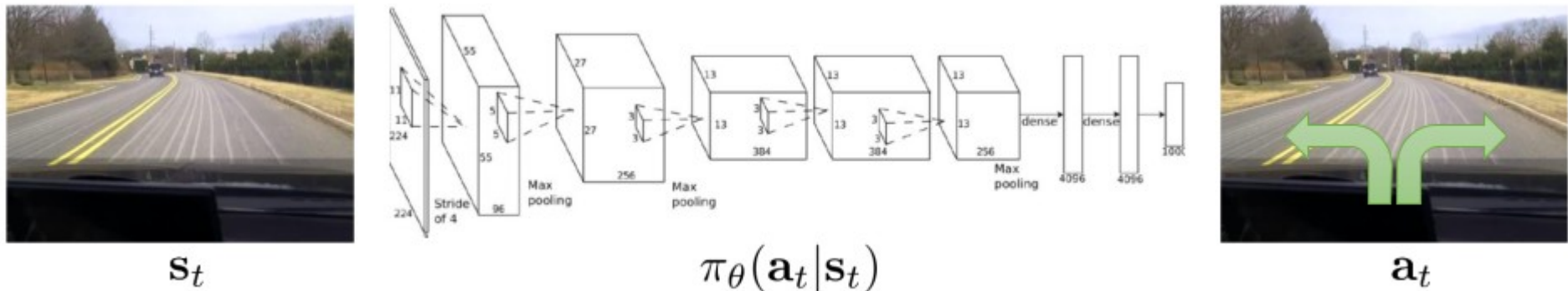
Sum over N trajectories $\{\tau^i\}_{i=0}^N$
For each step in the trajectory

REINFORCE algorithm:

1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



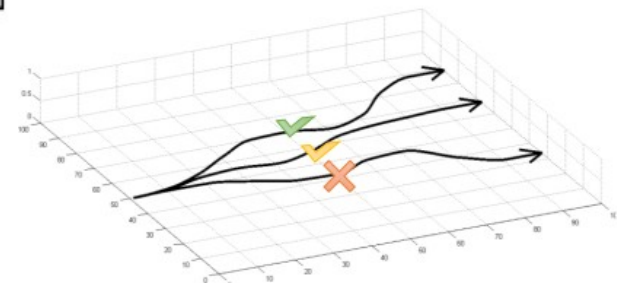
This is done with the deep neural network:



$$\text{recall: } J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, a_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(s_{i,t}, a_{i,t})$$

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) \left(\sum_{t=1}^T r(s_t, a_t) \right) \right]$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \left(\sum_{t=1}^T r(s_{i,t}, a_{i,t}) \right)$$



Intuition of Policy Gradient Maximum Likelihood

7. REINFORCE Algorithm

Essential Idea:

By maximizing $J(\theta)$ using this gradient, the policy learns to favor actions that lead to higher rewards.

$$\nabla_{\theta} J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau_i)$$

Intuition for the Gradient Term

Log Probability: $\log \pi_{\theta}(a_t | s_t)$

measures the likelihood of action a_t under the policy.

Gradient of Log Probability:

$$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

gives the direction to adjust θ to increase the likelihood of taking action a_t in state s_t

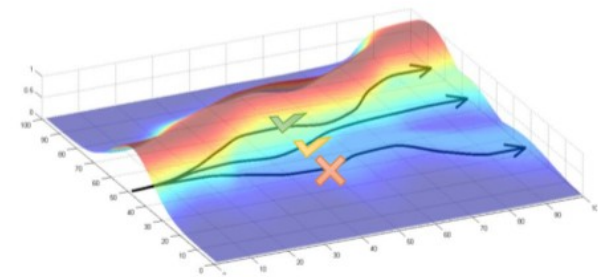
Weighted Return

$$G_t$$

scales the gradient by how "good" the outcome was.

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \underbrace{\nabla_{\theta} \log \pi_{\theta}(\tau_i)}_{\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})} r(\tau_i)$$



- good stuff is made more likely
- bad stuff is made less likely
- simply formalizes the notion of "trial and error"!

```

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import gymnasium as gym
import matplotlib.pyplot as plt

# Policy Network for FrozenLake
class PolicyNetwork(nn.Module):
    def __init__(self, state_size, action_size):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        self.fc2 = nn.Linear(128, action_size)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return torch.softmax(self.fc2(x), dim=-1)

```



```

# Function to generate trajectories
def generate_trajectory(env, policy_net):
    state, _ = env.reset()
    trajectory = []
    done = False

    while not done:
        # One-hot encode the discrete state
        state_tensor = torch.eye(env.observation_space.n)[state]
        # Get action probabilities
        action_probs = policy_net(state_tensor)
        action_dist = torch.distributions.Categorical(action_probs)
        action = action_dist.sample() # Sample an action
        log_prob = action_dist.log_prob(action) # Log-probability of action
        # Step in the environment
        next_state, reward, terminated, truncated, _ = env.step(action.item())
        done = terminated or truncated
        trajectory.append((state, action.item(), log_prob, reward))
        state = next_state # Update state
    return trajectory

```

REINFORCE Algorithm with Return Normalization

```
def reinforce(env, policy_net, optimizer, gamma=0.99, num_iterations=1000, N=10):
    losses = [] # To store the loss at each iteration
    avg_returns = [] # To track average return for monitoring

    for iteration in range(num_iterations):
        trajectories = []
        all_returns = []

        # Generate N trajectories
        for _ in range(N):
            trajectory = generate_trajectory(env, policy_net)
            trajectories.append(trajectory)

            # Compute returns for each trajectory
            returns = compute_returns(trajectory, gamma)
            all_returns.append(returns)

        # Normalize returns
        all_returns_flat = [G for returns in all_returns for G in returns]
        mean_return = np.mean(all_returns_flat)
        std_return = np.std(all_returns_flat) + 1e-8
        normalized_returns = [(returns - mean_return) / std_return for returns in all_returns]

        # Compute policy gradients
        policy_loss = 0
        for trajectory, returns in zip(trajectories, normalized_returns):
            for (state, action, log_prob, _), G in zip(trajectory, returns):
                policy_loss += -log_prob * G # Accumulate loss

        policy_loss /= N # Average over N trajectories

        # Update network parameters
        optimizer.zero_grad()
        policy_loss.backward()
        optimizer.step()

        # Track loss and average return
        losses.append(policy_loss.item())
        avg_return = np.mean([np.sum([reward for _, _, _, reward in traj]) for traj in trajectories])
        avg_returns.append(avg_return)

    # Logging
    if (iteration + 1) % 100 == 0:
        print(f"Iteration {iteration + 1}/{num_iterations}, Average Return: {avg_return:.2f}")
```

```
# Function to compute returns
def compute_returns(trajectory, gamma):
    returns = []
    G = 0
    for _, _, _, reward in reversed(trajectory):
        G = reward + gamma * G
        # Prepend to the list
        returns.insert(0, G)
    return returns
```

```
def run_human_mode(env, policy_net):
    state, _ = env.reset()
    env.render()
    done = False

    while not done:
        state_tensor = torch.tensor(state, dtype=torch.float32)
        action_probs = policy_net(state_tensor)
        # Choose action with highest probability
        action = torch.argmax(action_probs).item()

        state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        env.render()
```

Main Program

```
if __name__ == "__main__":
    # Settings
    train = True # Set to False to load the model and run in human mode
    env_name = "CartPole-v1"
    save_path = f"{env_name}_policy.pth"

    # Initialize environment and policy
    env = gym.make(env_name, render_mode="human" if not train else None)
    state_size = env.observation_space.shape[0] if env_name == "CartPole-v1" else env.observation_space.n
    action_size = env.action_space.n
    policy_net = PolicyNetwork(state_size, action_size)

    if train:
        optimizer = optim.Adam(policy_net.parameters(), lr=0.01)
        reinforce(env, policy_net, optimizer, gamma=0.99, num_iterations=1000, N=10, save_path=save_path)
    else:
        # Load trained model
        if os.path.exists(save_path):
            policy_net.load_state_dict(torch.load(save_path))
            print(f"Model loaded from {save_path}")
        else:
            print(f"No saved model found at {save_path}")
            exit()

    # Run environment in human mode
    run_human_mode(env, policy_net)
```

```

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import gymnasium as gym
import matplotlib.pyplot as plt

# Policy Network for FrozenLake
class PolicyNetwork(nn.Module):
    def __init__(self, state_size, action_size):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        self.fc2 = nn.Linear(128, action_size)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return torch.softmax(self.fc2(x), dim=-1)

```



```
class PolicyNetwork(nn.Module):
    def __init__(self, state_size, action_size):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, action_size)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return torch.softmax(self.fc3(x), dim=-1)
```

```
class PolicyNetwork(nn.Module):
    def __init__(self, state_size, action_size):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, action_size)
        # Dropout with 20% probability
        self.dropout = nn.Dropout(p=0.2)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x) # Apply dropout
        x = torch.relu(self.fc2(x))
        return torch.softmax(self.fc3(x), dim=-1)
```

```
class PolicyNetwork(nn.Module):
    def __init__(self, state_size, action_size):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        self.bn1 = nn.BatchNorm1d(128)
        self.fc2 = nn.Linear(128, 64)
        self.bn2 = nn.BatchNorm1d(64)
        self.fc3 = nn.Linear(64, action_size)

    def forward(self, x):
        # Normalize and activate
        x = torch.relu(self.bn1(self.fc1(x)))
        x = torch.relu(self.bn2(self.fc2(x)))
        return torch.softmax(self.fc3(x), dim=-1)
```

Adding More Hidden Layers

- Adding more hidden layers can help the network better approximate complex policies, especially if the state-action mapping is non-linear.
- Pros: Increased depth; learn more complex state-action relationships.
- Cons: May require more training time and data to converge.

Using Dropout

- Dropout helps to regularize the network and prevent overfitting
- Pros: Prevents overfitting in small datasets or environments with limited diversity.
- Cons: May slow down training slightly due to stochasticity introduced by dropout.

Batch Normalization

- Batch normalization can stabilize and speed up training by normalizing the input to each layer.
- Pros: Improves convergence speed and handles internal covariate shift.
- Cons: Adds computation overhead and requires larger batch sizes to be effective.

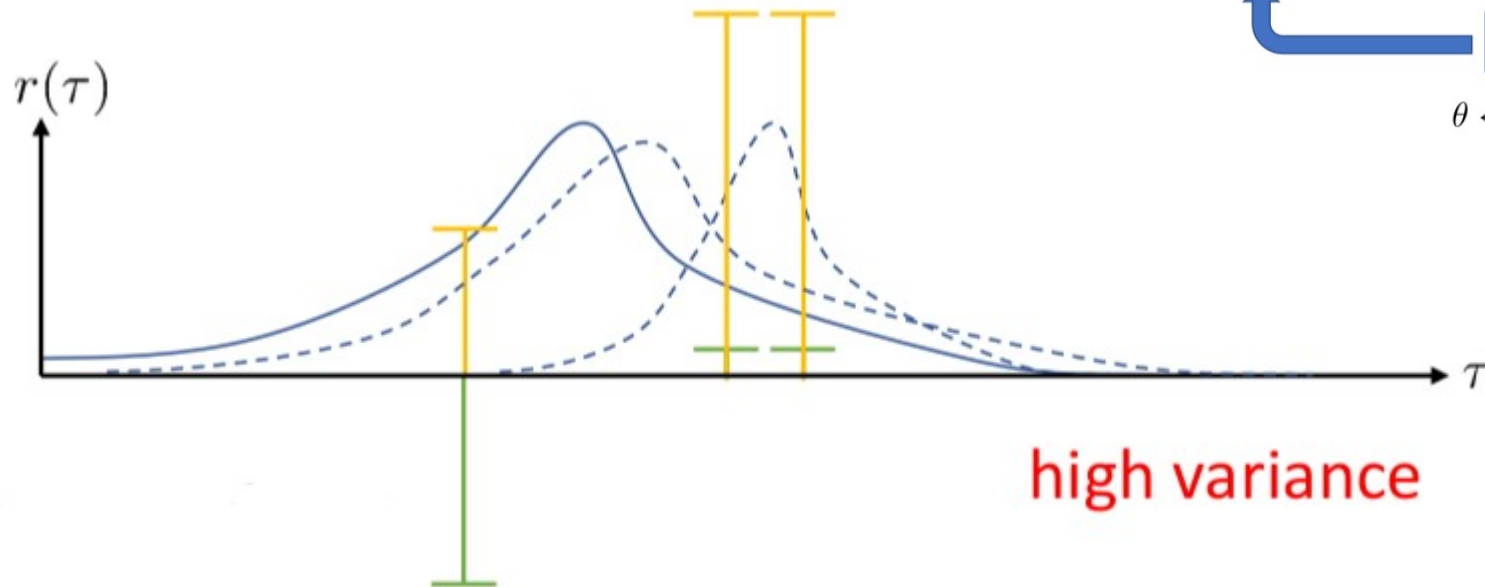
Comparison of Policy Gradient And DQN

Aspect	REINFORCE (Policy Gradient)	DQN (Q-Learning)
Goal	Maximize expected return by adjusting policy directly	Learn Q-values to inform policy indirectly
Input	Current state	Current state
Output	Action probabilities (policy)	Q-values for each action
Action Selection	Sample action based on probability	Choose action with max Q-value
Storage of Experience	Stores entire episodes, processes them immediately	Uses replay buffer to store and sample experiences
Gradient Update	Policy gradient with return	Temporal difference error with target Q-values
Suitability	Works well for continuous and stochastic policies	Works well for discrete and deterministic policies

What is wrong with Policy Gradient?

Policy Gradient has a high variance

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau)$$

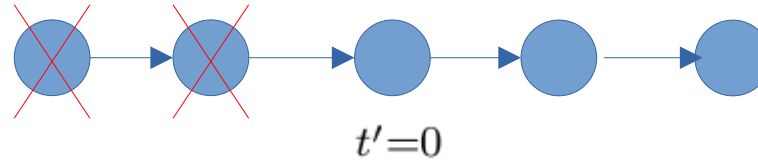


Generating samples to evaluate
the RL objective
Formalization of “trial and error”

Reducing variance: Policy gradient with Baseline And “Return to go”

Return-to-go: The return from a starting t

$$R_t = \sum_{t'=0}^{T-1} r(s_{t'}, a_{t'})$$

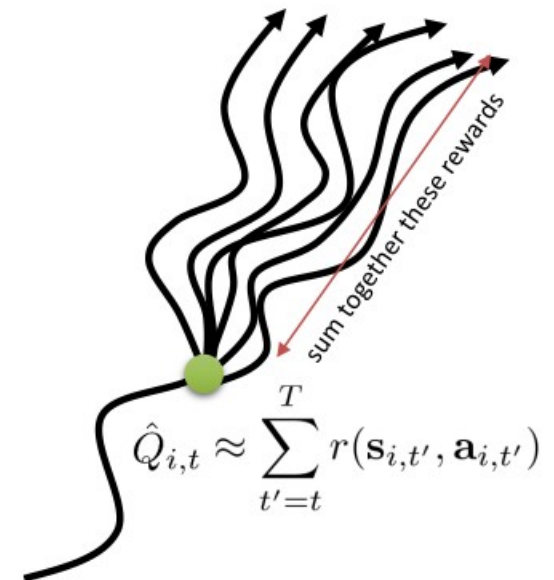
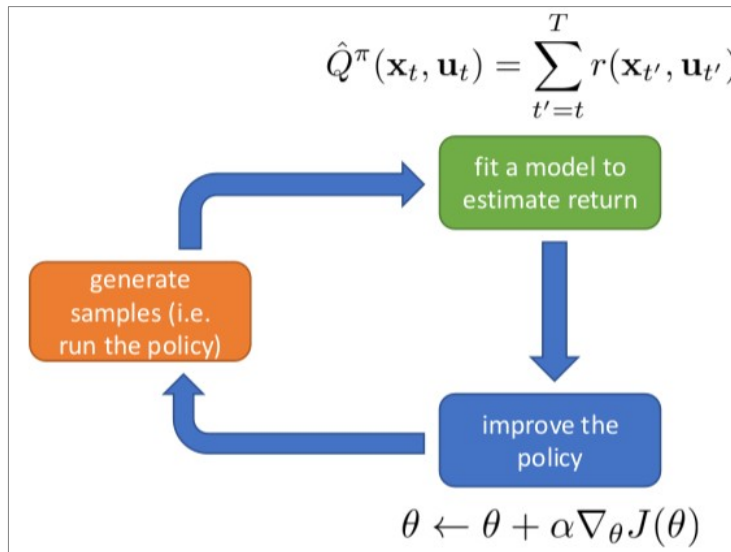


$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \underbrace{\left(\sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) \right)}_{\text{“reward to go”}}$$

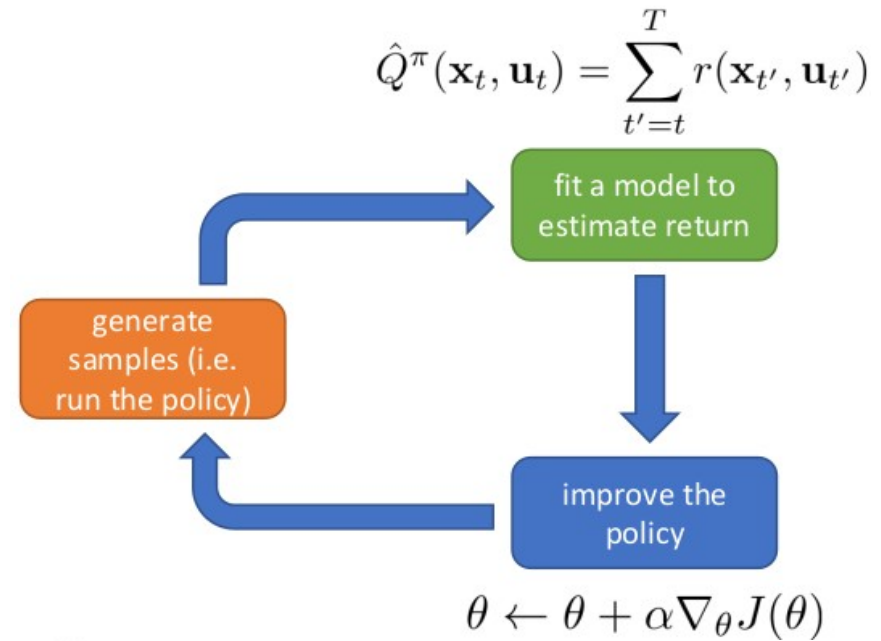
“reward to go”

$$\hat{Q}_{i,t} \quad \text{or} \quad R_t$$



REINFORCE algorithm:

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_{t=1}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i) \left(\sum_{t'=t}^T r(\mathbf{s}_{t'}^i, \mathbf{a}_{t'}^i) \right) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



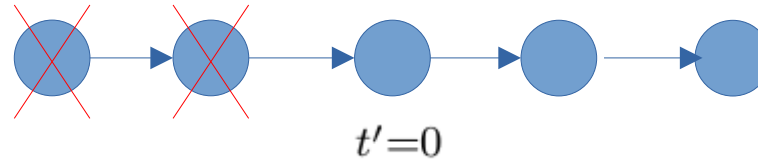
$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t}) \underbrace{\hat{Q}_{i,t}^\pi}_{\text{"reward to go"}}$$

"reward to go"

Reducing variance:
Policy gradient with
Baseline And “Return to go”

Return-to-go: The return from a starting t

$$R_t = \sum_{t'=0}^{T-1} r(s_{t'}, a_{t'})$$



$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_t \right]$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (R_t - b) \right]$$

Requires careful
math to justify that
this reduces
variance!

$$b = \frac{1}{N} \sum_{i=1}^N R(\tau^i)$$

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

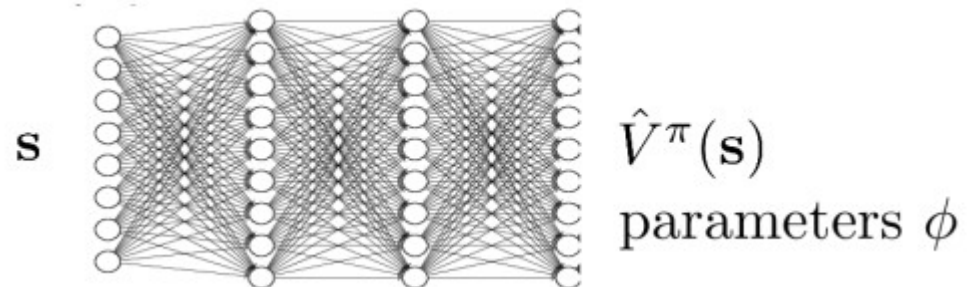
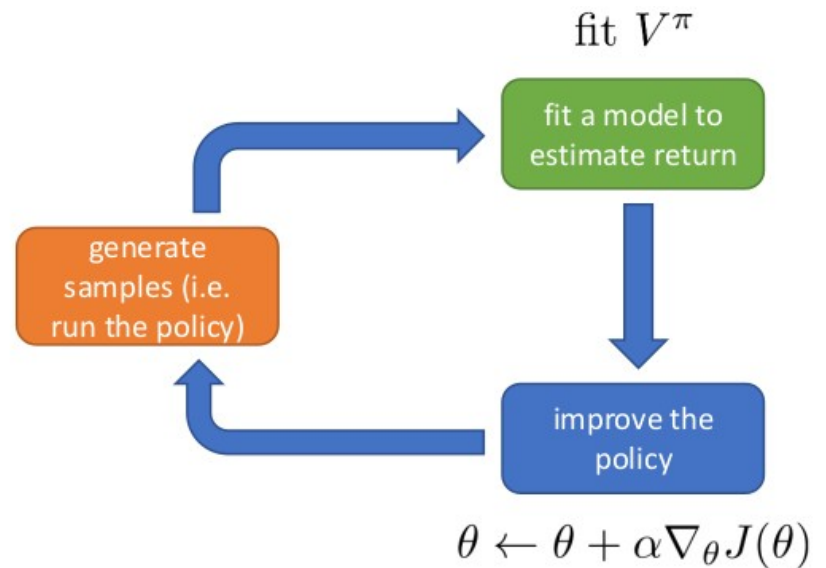
Policy gradient with Value function Baseline

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \underbrace{(R_t - V_{\phi}(s_t))}_{\text{Using the value function as the baseline}} \right]$$

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

$$J(\phi) = \frac{1}{N} \sum_{i=0}^N \sum_{t=0}^{T-1} (R_t - V_{\phi}(s_t))^2$$

$$\phi = \phi + \alpha \nabla_{\phi} J(\phi)$$



Two neural networks

Policy network parameterized by θ :

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (R_t - V_{\phi}(s_t)) \right]$$

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

Value network parameterized by ϕ :

$$J(\phi) = \frac{1}{N} \sum_{i=0}^N \sum_{t=0}^{T-1} (R_t - V_{\phi}(s_t))^2$$

$$\phi = \phi + \alpha \nabla_{\phi} J(\phi)$$

Step-by-Step Training Algorithm for REINFORCE with Baseline

1. Initialize the network parameters θ and the value network parameters ϕ
2. Generate N trajectories $\{\tau^i\}_{i=0}^N$ following policy: π_θ
3. Compute the return R_t or the discounted return G_t
4. Compute the gradients:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) (R_t - V_\phi(s_t)) \right]$$

5. Update the network parameter as $\theta = \theta + \alpha \nabla_\theta J(\theta)$
6. Compute the MSE of the value network:

$$J(\phi) = \frac{1}{N} \sum_{i=0}^N \sum_{t=0}^{T-1} (R_t - V_\phi(s_t))^2$$

7. Compute $\nabla_\phi J(\phi)$ update value network with gradient descent $\phi = \phi + \alpha \nabla_\phi J(\phi)$
8. Repeat steps 2 to 7 for several iterations


```

# Policy Network
class PolicyNetwork(nn.Module):
    def __init__(self, state_size,
action_size):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        self.fc2 = nn.Linear(128, action_size)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return torch.softmax(self.fc2(x),
dim=-1)

```

```

# Value Network
class ValueNetwork(nn.Module):
    def __init__(self, state_size):
        super(ValueNetwork,
self).__init__()
        self.fc1 = nn.Linear(state_size,
128)

        # Single output for state value
        self.fc2 = nn.Linear(128, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return self.fc2(x)

```

```

# Main Program
if __name__ == "__main__":
    # Settings
    env_name = "CartPole-v1" # Change to "FrozenLake-v1" for Frozen Lake
    env = gym.make(env_name)

    state_size = env.observation_space.shape[0] if env_name == "CartPole-v1" else env.observation_space.n
    action_size = env.action_space.n

    # Initialize networks
    policy_net = PolicyNetwork(state_size, action_size)
    value_net = ValueNetwork(state_size)

    # Optimizers
    policy_optimizer = optim.Adam(policy_net.parameters(), lr=0.01)
    value_optimizer = optim.Adam(value_net.parameters(), lr=0.01)

    # Train the networks
    reinforce_with_baseline(env, policy_net, value_net, policy_optimizer,
        value_optimizer, gamma=0.99, num_iterations=1000, N=10)

```

```

# REINFORCE with Baseline Algorithm
def reinforce_with_baseline(env, policy_net, value_net, policy_optimizer,
                           value_optimizer, gamma=0.99, num_iterations=1000, N=10):
    policy_losses = [] # To store the policy loss at each iteration
    value_losses = [] # To store the value loss at each iteration
    avg_returns = [] # To track average return for monitoring

    for iteration in range(num_iterations):
        trajectories = []
        all_returns = []

        # Generate N trajectories
        for _ in range(N):
            trajectory = generate_trajectory(env, policy_net)
            trajectories.append(trajectory)

            # Compute returns for each trajectory
            returns = compute_returns(trajectory, gamma)
            all_returns.append(returns)

        # Normalize returns
        all_returns_flat = [G for returns in all_returns for G in returns]
        mean_return = np.mean(all_returns_flat)
        std_return = np.std(all_returns_flat) + 1e-8
        normalized_returns = [(returns - mean_return) / std_return for returns in all_returns]

        # Compute policy and value losses
        policy_loss = 0
        value_loss = 0
        for trajectory, returns in zip(trajectories, normalized_returns):
            for (state, action, log_prob, _), G in zip(trajectory, returns):
                state_tensor = torch.tensor(state, dtype=torch.float32)
                baseline = value_net(state_tensor).squeeze(0) # Value network prediction

                # Advantage:  $(R_t - V_{\phi}(s_t))$ 
                advantage = G - baseline.item()

                # Policy loss
                policy_loss += -log_prob * advantage

                # Value loss (MSE)
                value_loss += advantage ** 2

        policy_loss /= N # Average over N trajectories
        value_loss /= N # Average over N trajectories

```

```

# Update policy network
policy_optimizer.zero_grad()
policy_loss.backward()
policy_optimizer.step()

```

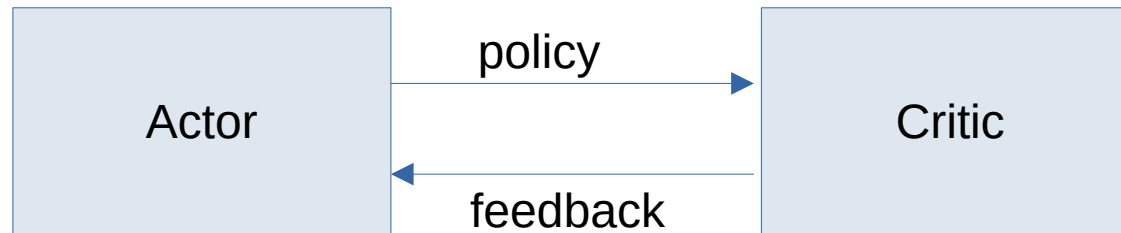
```

# Update value network
value_optimizer.zero_grad()
value_loss.backward()
value_optimizer.step()

```

3

The Actor -Critic Algorithms (AC, A2C, A3C)



decides which action to take;

Represents “policy” of agent;

Maps: states to actions

evaluates how good the action was

*Calculates the Advantage or error
between the predicted value and
actual reward*

- Like the policy-gradient with baseline, it reduces variance
- However, it also, helps to improve the policy iteratively.

The intuitive idea behind this approach:

- Have one part of the system (the actor) focus on learning the best actions to take in each situation,
- Another part (the critic) assesses those actions to provide feedback, guiding the learning process.

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (\underbrace{R_t - V_{\phi}(s_t)}_{\text{Return of a trajectory}}) \right]$$

New approximation:

$$R \approx r + \gamma V(s')$$

Immediate
reward

discounted
value of the
next state

$$\theta \text{ Actor Network: } \nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (r + \gamma V(s') - V_{\phi}(s_t))$$

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

$$\phi \text{ Critic Network: } J(\phi) = r + \gamma V_{\phi}(s') - V_{\phi}(s_t)$$

$$\phi = \phi + \alpha \nabla_{\phi} J(\phi)$$

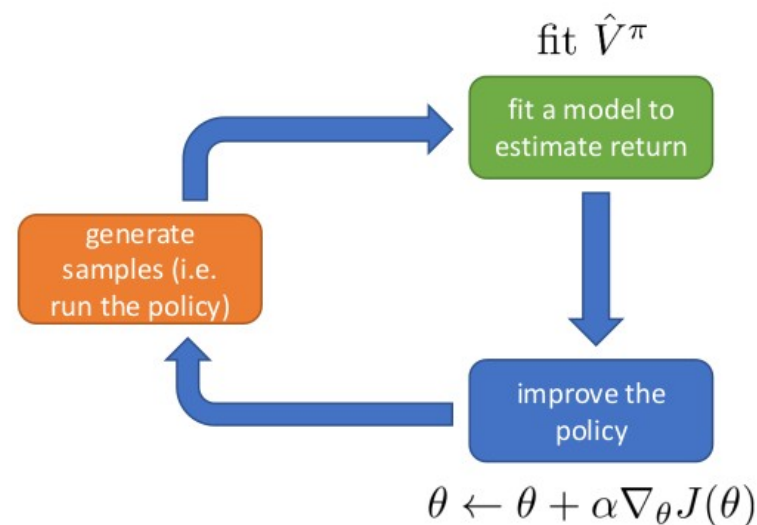
in the REINFORCE with baseline we use the full return of the trajectory
whereas in the actor-critic method we use the bootstrapped return.

Actor – critic algorithm

1. Initialize the actor network parameters θ and the critic network parameters ϕ
2. For N episodes, repeat step 3:
3. For each step in the episode, $t = 0, 1, 2, \dots, T - 1$
 - 3.1. Select an action using the policy, $a_t \sim \pi_\theta(s_t)$
 - 3.2. Take the action a_t in the state s_t , observe the reward r , and move to the next state s'_t
 - 3.3 Compute the policy gradients:
$$\nabla_\theta J(\theta) = \nabla_\theta \log \pi_\theta(a_t|s_t) \left(r + \gamma V_\phi(s'_t) - V_\phi(s_t) \right)$$
 - 3.4 Update the actor network parameter θ using gradient ascent:
$$\theta = \theta + \alpha \nabla_\theta J(\theta)$$
 - 3.5 Compute the loss of the critic network: $J(\phi) = r + \gamma V_\phi(s') - V_\phi(s_t)$
 - 3.6 Compute gradients $\nabla_\phi J(\phi)$ and update the critic network parameter ϕ using gradient descent:
$$\phi = \phi + \alpha \nabla_\phi J(\phi)$$

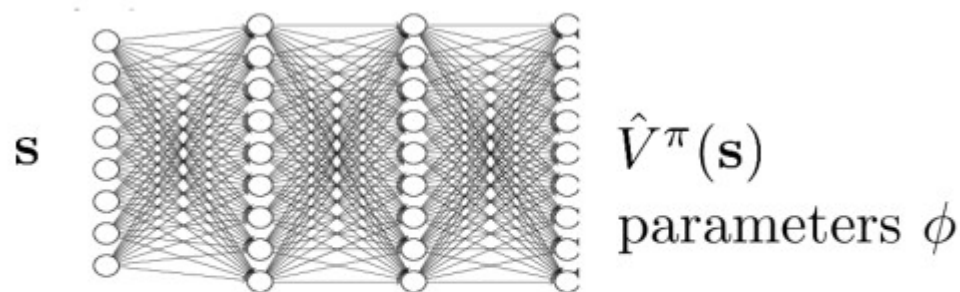
batch actor-critic algorithm:

1. sample $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ (run it on the robot)
2. fit $\hat{V}_\phi^\pi(\mathbf{s})$ to sampled reward sums
3. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



$$y_{i,t} \approx \sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})$$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$



$$V^\pi(\mathbf{s}_t) = \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t]$$

Policy evaluation

$$V^\pi(\mathbf{s}_t) = \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t]$$

$$J(\theta) = E_{\mathbf{s}_1 \sim p(\mathbf{s}_1)} [V^\pi(\mathbf{s}_1)]$$

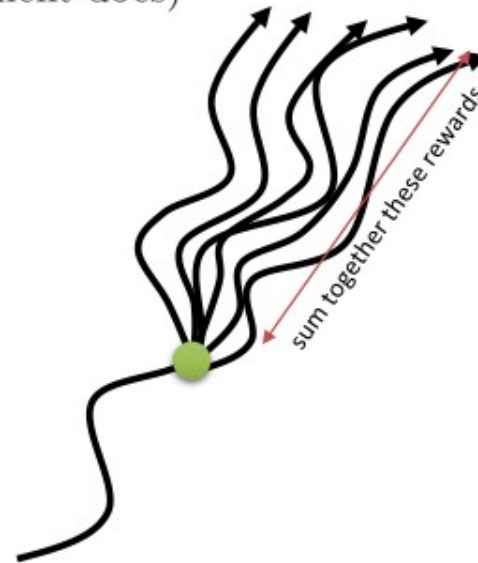
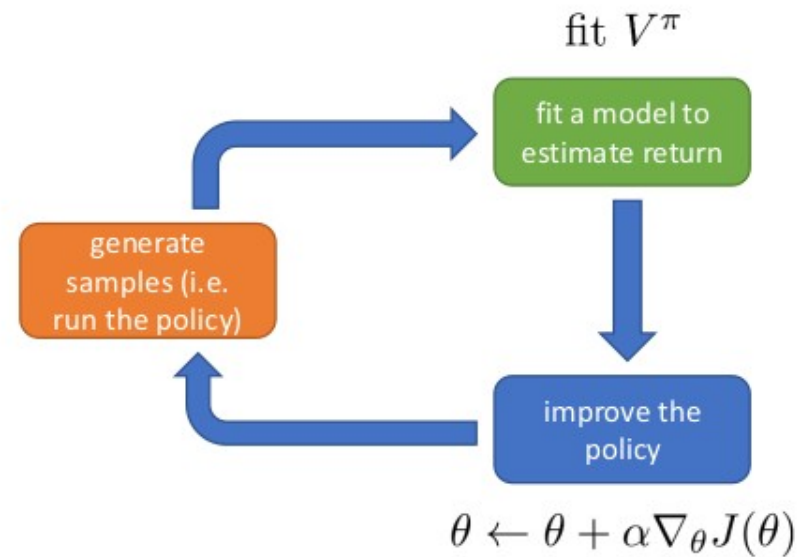
how can we perform policy evaluation?

Monte Carlo policy evaluation (this is what policy gradient does)

$$V^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$$

$$V^\pi(\mathbf{s}_t) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$$

(requires us to reset the simulator)



Monte Carlo evaluation with function approximation

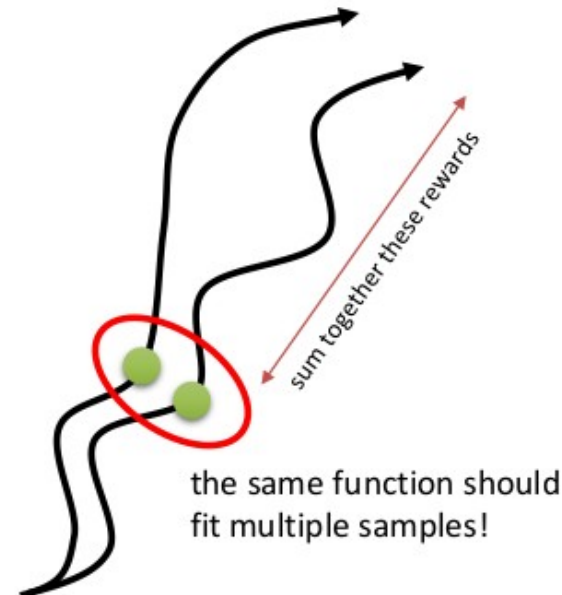
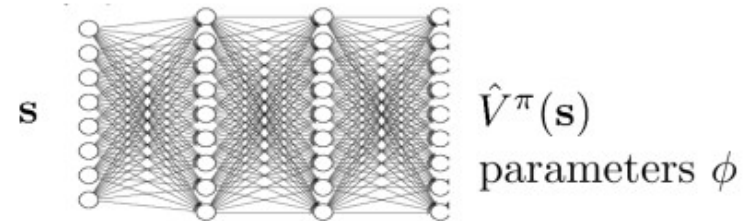
$$V^\pi(\mathbf{s}_t) \approx \sum_{t'=t}^T r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$$

not as good as this: $V^\pi(\mathbf{s}_t) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})$

but still pretty good!

training data: $\left\{ \left(\mathbf{s}_{i,t}, \underbrace{\sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})}_{y_{i,t}} \right) \right\}$

supervised regression: $\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$



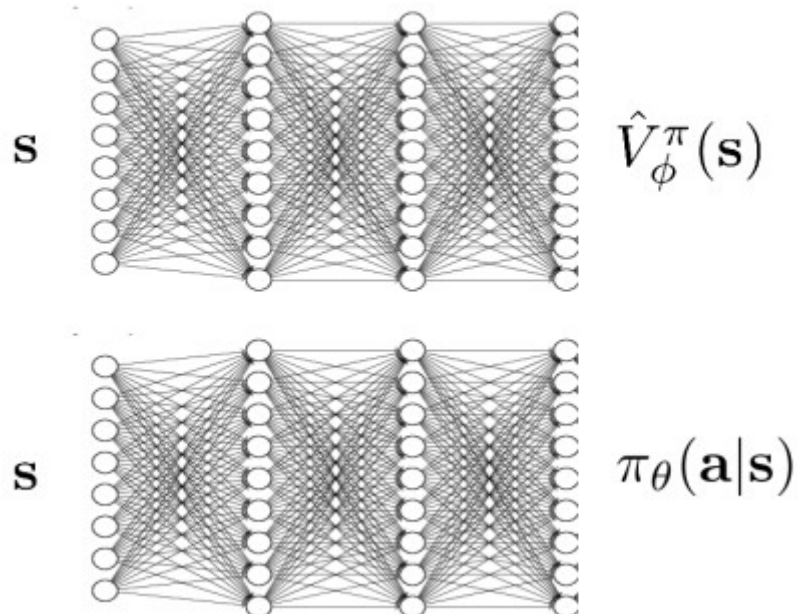
Actor (Policy) Network

```
class ActorNetwork(nn.Module):  
    def __init__(self, state_size, action_size):  
        super(ActorNetwork, self).__init__()  
        self.fc1 = nn.Linear(state_size, 128)  
        self.fc2 = nn.Linear(128, action_size)  
  
    def forward(self, x):  
        x = torch.relu(self.fc1(x))  
        return torch.softmax(self.fc2(x), dim=-1)
```

Critic (Value) Network

```
class CriticNetwork(nn.Module):  
    def __init__(self, state_size):  
        super(CriticNetwork, self).__init__()  
        self.fc1 = nn.Linear(state_size, 128)  
        # Single output for state value  
        self.fc2 = nn.Linear(128, 1)  
  
    def forward(self, x):  
        x = torch.relu(self.fc1(x))  
        return self.fc2(x)
```

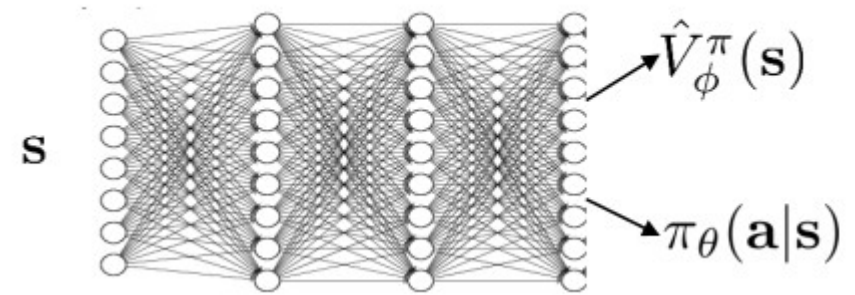
two network design



+ simple & stable

- no shared features between actor & critic

shared network design



```
# Main Program
if __name__ == "__main__":
    # Settings
    train = True # Set to False to load the model and run in human mode
    env_name = "CartPole-v1"
    save_path = f"{env_name}_actor_critic.pth"

    # Initialize environment
    env = gym.make(env_name, render_mode="human" if not train else None)
    state_size = env.observation_space.shape[0] if env_name == "CartPole-v1" else env.observation_space.n
    action_size = env.action_space.n

    # Initialize networks
    actor_net = ActorNetwork(state_size, action_size)
    critic_net = CriticNetwork(state_size)

    # Optimizers
    actor_optimizer = optim.Adam(actor_net.parameters(), lr=0.01)
    critic_optimizer = optim.Adam(critic_net.parameters(), lr=0.01)

    if train:
        # Train the networks
        actor_critic(env, actor_net, critic_net, actor_optimizer, critic_optimizer,
                    gamma=0.99, num_episodes=1000, save_path=save_path)
    else:
        # Load trained models
        if os.path.exists(save_path):
            checkpoint = torch.load(save_path)
            actor_net.load_state_dict(checkpoint["actor"])
            critic_net.load_state_dict(checkpoint["critic"])
            print(f"Model loaded from {save_path}")
        else:
            print(f"No saved model found at {save_path}")
            exit()

    # Run in prediction mode
    run_human_mode(env, actor_net)
```

Actor-Critic Algorithm

```
def actor_critic(env, actor_net, critic_net, actor_optimizer, critic_optimizer, gamma=0.99,
                 num_episodes=1000, save_path=None):
    actor_losses = [] # To store actor losses
    critic_losses = [] # To store critic losses
    avg_returns = [] # To track average return per episode
    for episode in range(num_episodes):
        state, _ = env.reset()
        done = False
        trajectory = []
        total_reward = 0
        while not done:
            # Convert state to tensor
            state_tensor = torch.tensor(state, dtype=torch.float32)
            # Actor chooses action
            action_probs = actor_net(state_tensor)
            action_dist = torch.distributions.Categorical(action_probs)
            action = action_dist.sample()
            log_prob = action_dist.log_prob(action) # Log-probability of action
            # Take action in environment
            next_state, reward, terminated, truncated, _ = env.step(action.item())
            done = terminated or truncated
            total_reward += reward
            # Convert next state to tensor
            next_state_tensor = torch.tensor(next_state, dtype=torch.float32)
            # Critic computes values
            value_s = critic_net(state_tensor)
            value_s_prime = critic_net(next_state_tensor).detach() # Detach to avoid gradient flow
            # Compute advantage for actor
            td_error = reward + gamma * value_s_prime - value_s # Temporal Difference (TD) error
            actor_loss = -log_prob * td_error.item() # Policy gradient loss
            # Update actor network
            actor_optimizer.zero_grad()
            actor_loss.backward()
            actor_optimizer.step()
            # Compute critic loss (TD error squared)
            critic_loss = td_error ** 2
            # Update critic network
            critic_optimizer.zero_grad()
            critic_loss.backward()
            critic_optimizer.step()
            # Save losses
            actor_losses.append(actor_loss.item())
            critic_losses.append(critic_loss.item())
            # Move to the next state
            state = next_state
```

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A(s, a)$$

$$A(s, a) = r + \gamma V(s') - V(s)$$

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(r + \gamma V_{\phi}(s') - V_{\phi}(s) \right)$$

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\underbrace{r + \gamma V_{\phi}(s'_t)}_{Q(s, a)} - \underbrace{V_{\phi}(s_t)} \right)$$

Advantage Actor-critic (A2C & A3C)

Advantage function:

$$A(s, a) = Q(s, a) - V(s)$$

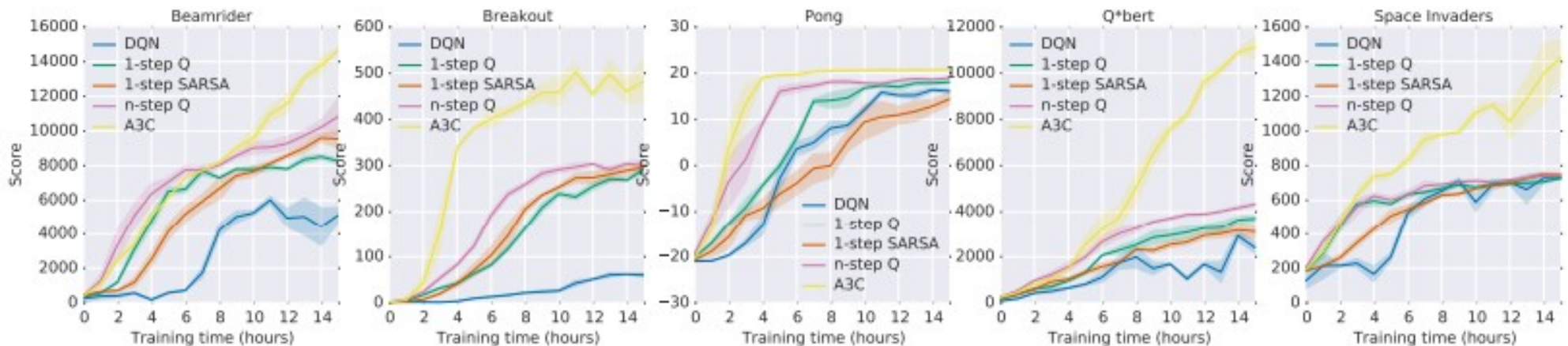
measures how much better (or worse) an action is compared to the average action at a given state.

$$Q(s, a) \approx r + \gamma V(s')$$

$$A(s, a) = r + \gamma V(s') - V(s)$$

The policy gradient (the direction in which the policy should be updated) is proportional to the advantage function:

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot A(s, a)]$$



<https://arxiv.org/abs/1602.01783>

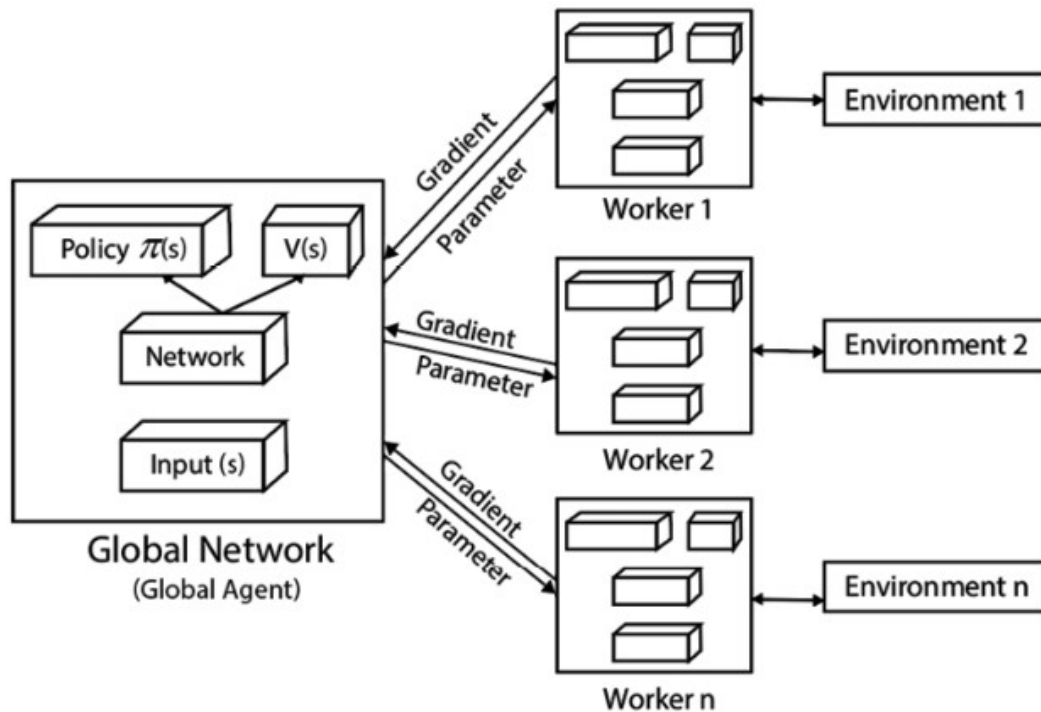
Variance Reduction: Using the advantage function helps reduce the variance in the policy gradient estimates

$$\text{TD error: } \delta = r + \gamma V(s') - V(s)$$

Advantage Actor – critic algorithm (same algorithm as before)

1. Initialize the actor network parameters θ and the critic network parameters ϕ
2. For N episodes, repeat step 3:
3. For each step in the episode, $t = 0, 1, 2, \dots, T - 1$
 - 3.1. Select an action using the policy, $a_t \sim \pi_\theta(s_t)$
 - 3.2. Take the action a_t in the state s_t , observe the reward r , and move to the next state s'_t
 - 3.3 Compute the policy gradients:
$$\nabla_\theta J(\theta) = \nabla_\theta \log \pi_\theta(a_t|s_t) \left(r + \gamma V_\phi(s'_t) - V_\phi(s_t) \right)$$
 - 3.4 Update the actor network parameter θ using gradient ascent:
$$\theta = \theta + \alpha \nabla_\theta J(\theta)$$
 - 3.5 Compute the loss of the critic network: $J(\phi) = r + \gamma V_\phi(s') - V_\phi(s_t)$
 - 3.6 Compute gradients $\nabla_\phi J(\phi)$ and update the critic network parameter ϕ using gradient descent:
$$\phi = \phi + \alpha \nabla_\phi J(\phi)$$

Goal: Efficiently learn an optimal policy by combining diverse experiences from asynchronously operating agents.



- Asynchronous: Multiple worker agents interact with their own copies of the environment in parallel. These agents learn independently and report updates to a central global agent.
- Advantage: Uses the advantage function to improve the actor's policy by estimating how much better an action is compared to the baseline (value function)
- Actor-Critic: Combines an actor network (to estimate the policy) and a critic network (to evaluate the policy) for both worker agents and the global agent.

- Global Agent: Aggregates the experience and updates shared parameters.
- Worker Agents: Explore using different policies, collect experiences, and asynchronously send updates to the global agent.

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(r + \gamma V_{\phi}(s'_t) - V_{\phi}(s_t) \right) + \underbrace{\beta H(\pi(s))}_{\text{Entropy}}$$

Implementaitons of AC methods.

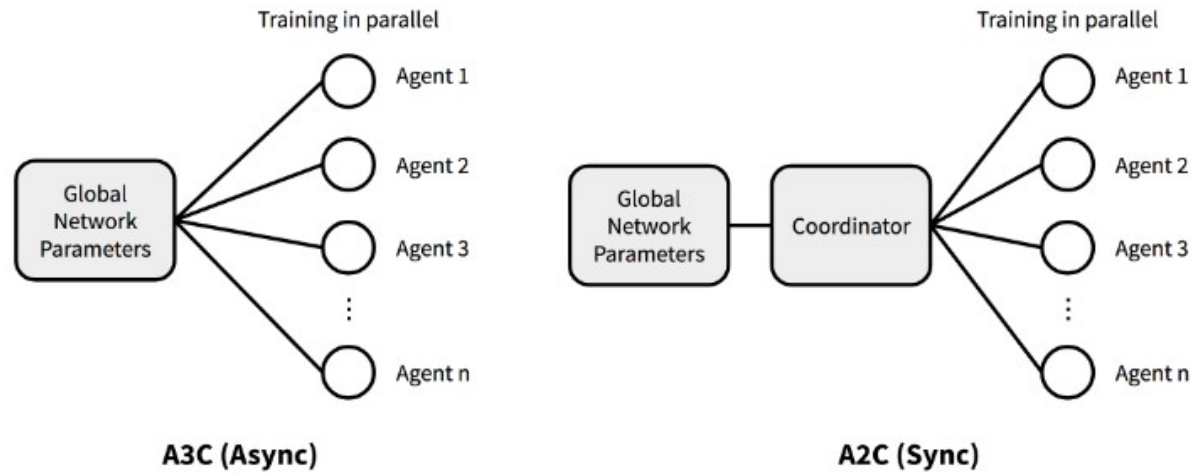
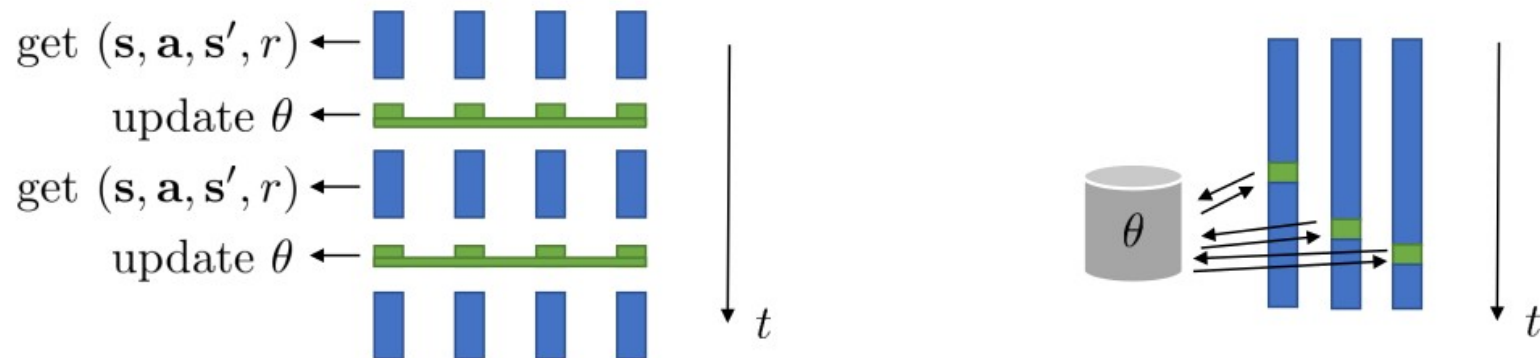


Fig. 2. The architecture of A3C versus A2C.



Asynchronous Advantage Actor – critic algorithm (A3C)

1. The worker agent interacts with their own copies of the environment.
2. Each worker follows a different policy and collects the experience.
3. Next, the worker agents compute the losses of the actor and critic networks.
4. After computing the loss, they calculate gradients of the loss, and send those gradients to the global agent asynchronously.
5. The global agent updates their parameters with the gradients received from the worker agents.
6. Now, the updated parameter from the global agent will be sent to the worker agents periodically.

```
# Actor (Policy) Network
class ActorNetwork(nn.Module):
    def __init__(self, state_size, action_size):
        super(ActorNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        self.fc2 = nn.Linear(128, action_size)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return torch.softmax(self.fc2(x), dim=-1)
```

```
# Critic (Value) Network
class CriticNetwork(nn.Module):
    def __init__(self, state_size):
        super(CriticNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        # Single output for state value
        self.fc2 = nn.Linear(128, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return self.fc2(x)
```

```
# Main A3C Algorithm
def a3c(env_name, num_workers, gamma=0.99, num_episodes=5000, update_freq=5, lr=0.001, save_path=None):
    env = gym.make(env_name)
    state_size = env.observation_space.shape[0]
    action_size = env.action_space.n

    # Global networks
    actor_net = ActorNetwork(state_size, action_size)
    critic_net = CriticNetwork(state_size)
    actor_net.share_memory()
    critic_net.share_memory()

    # Optimizers
    actor_optimizer = optim.Adam(actor_net.parameters(), lr=lr)
    critic_optimizer = optim.Adam(critic_net.parameters(), lr=lr)

    # Shared memory for losses
    actor_losses = mp.Manager().list()
    critic_losses = mp.Manager().list()

    # Spawn worker processes
    processes = []
    for worker_id in range(num_workers):
        process = mp.Process(target=worker_process, args=(actor_net, critic_net, actor_optimizer,
                                                         critic_optimizer, env_name, gamma, worker_id, update_freq, actor_losses, critic_losses))
        processes.append(process)
        process.start()

    # Wait for all processes to complete
    for process in processes:
        process.join()

    # Save trained parameters
    if save_path:
        torch.save({
            "actor": actor_net.state_dict(),
            "critic": critic_net.state_dict()
        }, save_path)
        print(f"Model saved to {save_path}")
```

```

def worker_process(actor_net, critic_net, actor_optimizer, critic_optimizer, env_name,
                  gamma, worker_id, update_freq, actor_losses, critic_losses):
    env = gym.make(env_name)
    local_actor = ActorNetwork(env.observation_space.shape[0], env.action_space.n)
    local_critic = CriticNetwork(env.observation_space.shape[0])

    # Sync local networks with global networks
    local_actor.load_state_dict(actor_net.state_dict())
    local_critic.load_state_dict(critic_net.state_dict())

    state, _ = env.reset()
    done = False
    states, actions, rewards, log_probs = [], [], [], []

    while True:
        # Step through the environment
        state_tensor = torch.tensor(state, dtype=torch.float32)
        policy = local_actor(state_tensor)
        action_dist = torch.distributions.Categorical(policy)
        action = action_dist.sample()
        log_prob = action_dist.log_prob(action)

        next_state, reward, terminated, truncated, _ = env.step(action.item())
        done = terminated or truncated
        states.append(state)
        actions.append(action)
        rewards.append(reward)
        log_probs.append(log_prob)

        state = next_state
        if done or len(states) >= update_freq:
            # Compute returns
            if not done:
                with torch.no_grad():
                    next_value = local_critic(torch.tensor(next_state, dtype=torch.float32))
                    rewards[-1] += gamma * next_value.item()

            returns = []
            G = 0
            for reward in reversed(rewards):
                G = reward + gamma * G
                returns.insert(0, G)
            returns = torch.tensor(returns, dtype=torch.float32)

```

```
# Calculate advantages
states_tensor = torch.tensor(states, dtype=torch.float32)
values = local_critic(states_tensor).squeeze(1)
advantages = returns - values

# Actor loss
actor_loss = -(torch.stack(log_probs) * advantages.detach()).mean()

# Critic loss
critic_loss = advantages.pow(2).mean()

# Track losses
actor_losses.append(actor_loss.item())
critic_losses.append(critic_loss.item())

# Update global networks
actor_optimizer.zero_grad()
critic_optimizer.zero_grad()
actor_loss.backward()
critic_loss.backward()

for global_param, local_param in zip(actor_net.parameters(), local_actor.parameters()):
    global_param._grad = local_param.grad
for global_param, local_param in zip(critic_net.parameters(), local_critic.parameters()):
    global_param._grad = local_param.grad

actor_optimizer.step()
critic_optimizer.step()

# Sync local networks with global networks
local_actor.load_state_dict(actor_net.state_dict())
local_critic.load_state_dict(critic_net.state_dict())

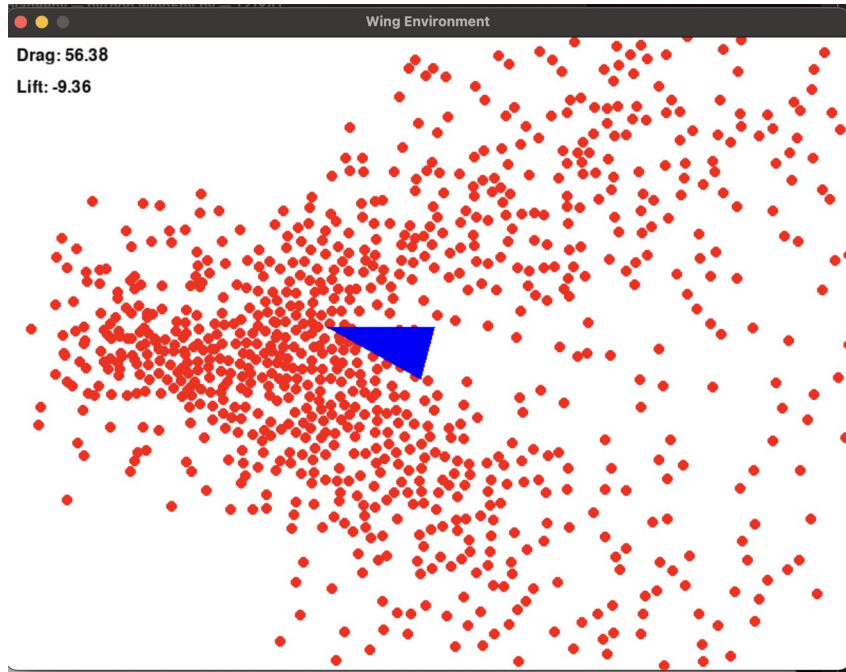
# Reset buffers
states, actions, rewards, log_probs = [], [], [], []

if done:
    state, _ = env.reset()
    done = False
```

4

Wing design with custom
particle wind tunnel
(we will take a look at this in the lab)

Custom Wing design

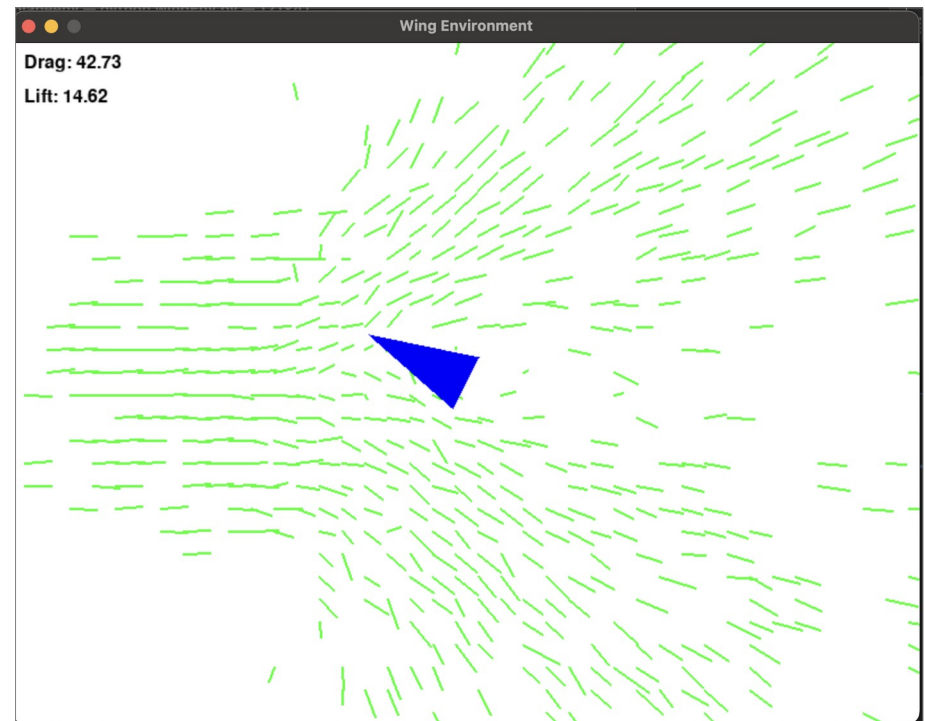
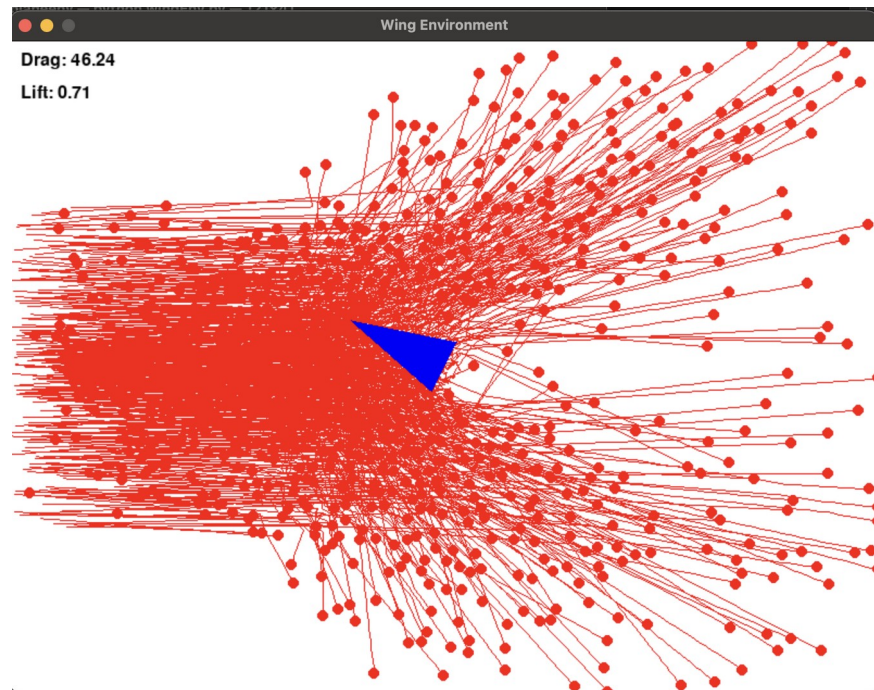


Environment that has:

- Particle based wind dynamics
- A simple parameterized polygon

Objective:

- show how RL can perform design tasks




```

class Particle:
    def __init__(self, x, y, radius, velocity, theta, screen_width, screen_height):
        self.x = x
        self.y = y
        self.radius = radius
        self.theta = theta
        self.velocity = velocity
        self.screen_width = screen_width
        self.screen_height = screen_height
        self.vel_x = self.velocity * math.cos(self.theta)
        self.vel_y = self.velocity * math.sin(self.theta)
        self.path = [] # For visualizing the path of the particle

    def move(self):
        self.x, self.y = move_particle(self.x, self.y, self.vel_x, self.vel_y)
        self.path.append((self.x, self.y))
        if len(self.path) > 100: # Limit path history to 100 positions
            self.path.pop(0)

    def handle_collision_with_walls(self):
        return check_wall_collision(self.x, self.y, self.screen_width, self.screen_height, self.radius)

    def handle_collision_with_particle(self, other_particle):
        self.vel_x, self.vel_y, other_particle.vel_x, other_particle.vel_y = handle_particle_collision(
            self.x, self.y, self.vel_x, self.vel_y,
            other_particle.x, other_particle.y, other_particle.vel_x, other_particle.vel_y, self.radius
        )

```

```

.
├── deltaWing.py
├── dqnWing.py
├── particleWind.py
└── wingEnv.py

```

```

class ParticleWindSimulator:
    def __init__(self, num_particles_per_group=5, deltat=1, use_nozzle=True, width=800, height=600, grid_size=20):
        self.WIDTH = width
        self.HEIGHT = height
        self.num_particles_per_group = num_particles_per_group
        self.particle_radius = 5
        self.particles = []
        self.use_nozzle = use_nozzle
        self.deltat = deltat
        self.last_release_time = time.time()
        self.grid_size = grid_size
        self.grid = np.zeros((self.WIDTH // grid_size, self.HEIGHT // grid_size, 2)) # For averaging velocity
        self.grid_paths = {} # To store tracks of mean velocities
        self.drag = 0 # Track drag force
        self.lift = 0 # Track lift force
        self.show_paths = False
        self.use_grid = False
        self.show_grid_paths = False

```

Next time:
Other on-policy methods
using stable Baseline3

Mitigating Training instability

https://stable-baselines3.readthedocs.io/en/master/guide/rl_tips.html

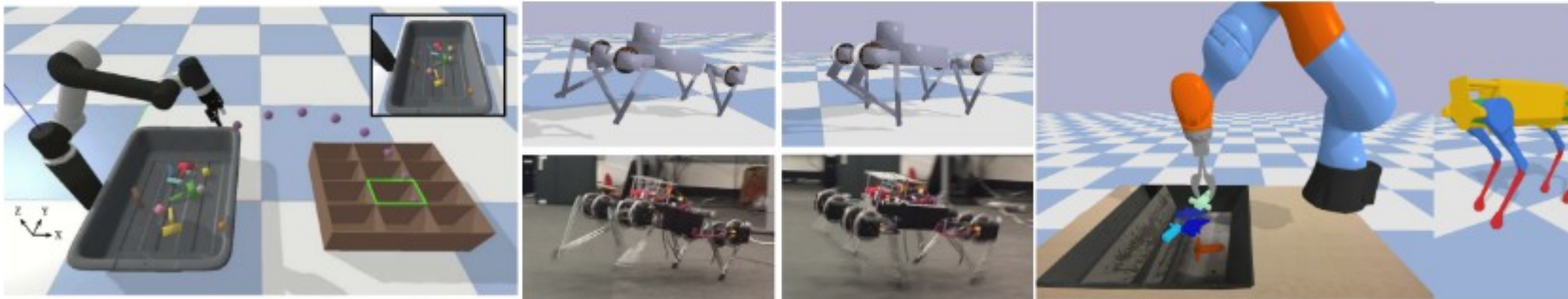
One last limitation of RL is the instability of training. That is to say, you can observe during training a huge drop in performance. This behavior is particularly present in `DDPG`, that's why its extension `TD3` tries to tackle that issue. Other method, like `TRPO` or `PPO` make use of a *trust region* to minimize that problem by avoiding too large update.

<https://github.com/bulletphysics/bullet3>

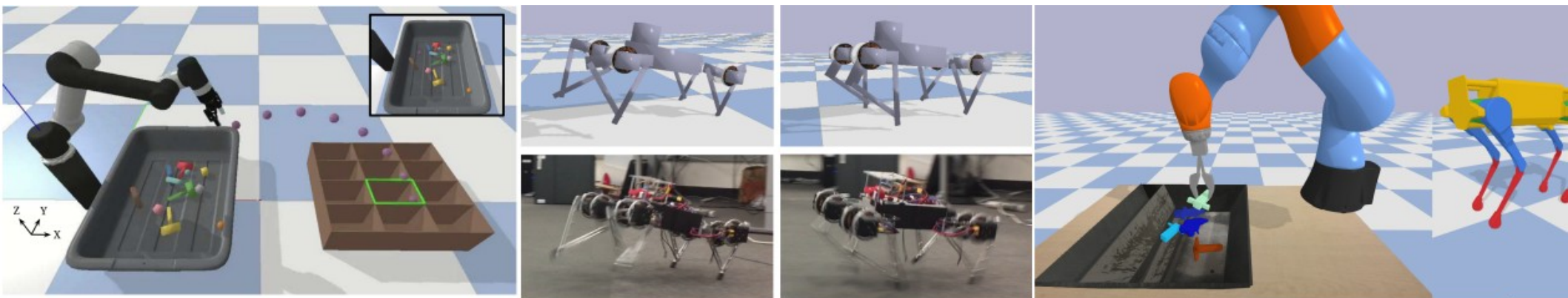


Bullet Physics SDK

This is the official C++ source code repository of the Bullet Physics SDK: real-time collision detection and multi-physics simulation for VR, games, visual effects, robotics, machine learning etc.



More realistic



Bullet Real-Time Physics Simulation

Home of Bullet and PyBullet:
physics simulation for games,
visual effects, robotics and
reinforcement learning.

RECENT POSTS

Kubric: a scalable dataset
generator

NeuralSim: Augmenting
Differentiable Simulators with
Neural Networks

PyBullet in a colab

Learning Agile Robotic
Locomotion Skills by Imitating
Animals

Assistive gym @ ICRA, 2020

SELECTED VIDEO

BLOG

FORUM

GITHUB

PYBULLET QUICKSTART GUIDE

DOWNLOAD

DOCUMENTATION

FORUM



KUBRIC: A SCALABLE DATASET GENERATOR

🕒 MARCH 21, 2022

Kubric is an open-source Python framework that interfaces with PyBullet and Blender to generate photo-realistic scenes, with rich annotations, and seamlessly scales to large jobs distributed over thousands of machines, and generating TBs of data. Kubric can generate semi-realistic synthetic multi-object videos with rich annotations such as instance segmentation masks, depth maps, and optical flow.

LAST TOPICS POSTS

[I C A R U S - summer 2024 newsletter](#)

...some high drag settings (attached) which you can load to try for yourself... ..

by skinduptruk

May 21, 2024 7:26 am

[crash at](#)

[btConvexHullShape::localGetSupportingVertexWithoutMargin](#)

I don't use btStaticPlaneShape but in the latest update this error has been fixed. geometry ...

by shallshe

January 12, 2024 1:28 am

[SDF link collisions change robot behavior, despite no interaction with robot](#)

I created a robot that sends motor values to joints at each step of the ...

<https://pybullet.org/>